PrivAGM: Secure Construction of Differentially Private Directed Attributed Graph Models on Decentralized Social Graphs

***Abstract*—Decentralized social graphs, where no single entity possesses access to the entire graph, and each user maintains only a restricted view of the graph, offer valuable applications. However, collecting local views for analytics raises significant privacy concerns due to the sensitive social relationships they capture. To address this, a canonical approach involves fitting a differentially private generative graph model to the input decentralized social graph, sampling a synthetic graph that serves as a proxy for analysis. Existing solutions, however, often fail to capture the inherent directionality of edges and attribute-edge correlations when dealing with decentralized directed social graphs, leading to synthetic graphs with poor utility. To bridge this gap, we present PrivAGM, an innovative solution that harnesses the synergies between differential privacy, secure multiparty computation, and generative graph models, enabling the secure construction of differentially private directed attributed graph models on decentralized social graphs while ensuring the preservation of individual privacy. Our comprehensive validation on three real-world directed social graph datasets, utilizing commonly used graph metrics, unequivocally demonstrates that PrivAGM outperforms the state-of-the-art methods, providing significantly higher utility in synthetic graphs.**

1. Introduction

With the continuous advancements in graph analytics, a multitude of valuable applications can stem from the in-depth exploration of a social graph, such as the discovery of social roles [1], mining of interests [2], and prediction of disease outbreaks [3]. However, social graph analytics becomes quite challenging when the graph is not accessible to a single entity and exists in a *decentralized* manner. In this scenario, each user possesses only a limited local view comprising its attributes and neighbor list within the overall social graph, and the complete social graph emerges from the collective views of all users. Decentralized social graphs can manifest in various practical applications [4]–[8]. For example, the phone network exemplifies a decentralized social graph, wherein each user maintains their own profile (i.e., attributes) and contact list (i.e., neighbor list), and the combination of all users' profiles and contact lists constitutes the social graph [5].

In the context of decentralized social graphs, the collection of individual users' local views for analytics can lead to significant privacy concerns, particularly when these local views contain sensitive information about social interactions, political preferences, and religious affiliations [4]–[7], [9]. If users' local views are not sufficiently protected, it will lead to understandable hesitancy from them to participate in such analytics. Thus, it is crucial to incorporate security measures into analytics over decentralized social graphs from the outset, ensuring the development and deployment of valuable applications without compromising users' privacy.

Privacy-preserving analysis over decentralized social graphs has received significant research attention in recent years [4]–[6], [9]–[12]. The bulk of existing solutions [4], [6], [9], [11] have adopted an *output perturbation* approach, where the statistics extracted from the decentralized social graph are perturbed with noise for privacy protection. Despite its popularity, output perturbation suffers from a major deficiency that limits its practical application. Specifically, as more statistics are collected, the total amount of revealed information monotonically increases, eventually reaching a threshold where collecting any new statistics leads to excessive overall privacy risk [13].

A natural mitigation involves privately fitting a generative graph model to the decentralized social graph using the framework of differential privacy (DP) [14]. The differentially private generative graph model is then employed to sample a synthetic graph that emulates the crucial characteristics of the decentralized graph and serves as a reliable proxy for conducting social graph analysis tasks. Based on the post-processing invariance property of DP [14], the synthetic graph remains differentially private. Hay *et al.* [15] highlight three important motivations for producing synthetic graphs: (1) Certain analyses require running complex algorithms or simulations over actual social graph data, rather than relying solely on statistics. (2) Synthetic graphs facilitate "exploratory, open-ended, and iterative" analysis, which can be challenging to accommodate directly on an input private graph with limited privacy budget under DP. (3) It eliminates the need to share proprietary analysis details with the social graph owner. Hence, the objective of this paper is to securely construct differentially private generative graph models on decentralized social graphs.

Existing solutions. In the literature, the construction of differentially private generative graph models has garnered considerable attention. However, most existing works (e.g., [3], [7], [16], to list a few) focus on a *centralized* model, where the complete graph is available to a single entity and processed in the plaintext domain. There has been little research [5], [10], [12], [17] on the *decentralized* model. Additionally, works like [5], [12], [17] concentrate solely on modeling graph structure, disregarding vertex attributes and their correlations with the graph structure. Nevertheless, real-world social graphs possess vertex attributes and exhibit such correlations. For example, users often have various attributes like age, gender, sexual preference, and they are

known to exhibit homophily, wherein users with similar attributes tend to form connections [18]. These correlations are leveraged in various analyses; for instance, in the field of relational machine learning, they are used to predict missing or future attribute values [3]. The work most closely related to ours is AsgLDP [10], which aims to construct a differentially private attributed graph model (AGM) on the decentralized social graph.

However, AsgLDP as well as other works [5], [12], [17] focus primarily on idealized decentralized *undirected* social graphs, where each user possesses its *complete* neighbor list. In practice, however, decentralized social graphs are more commonly *directed*, where each user possesses an *incomplete* neighbor list consisting of only its "one way relationships". For instance, in phone networks, it is possible that Peter's contact list contains Tom's phone number, while Tom's contact list does not contain Peter's phone number. Another example is the Diaspora application [8], which incorporates the concept of "asymmetric following", enabling users to follow others without requiring mutual consent. Therefore, existing works are not effective in handling such a more practical scenario. Additionally, all previous works do not consider the inherent directionality of the edges when constructing the generative graph model, leading to synthetic graphs with poor utility when dealing with directed graphs. As a consequence, the secure construction of differentially private directed AGMs (dAGMs) on decentralized directed social graphs remains an open problem.

Our contributions. In light of the above, in this paper, we design, implement, and evaluate PrivAGM. By ingeniously bridging the gap between DP, secure multiparty computation (MPC), and generative graph models, PrivAGM enables the secure construction of differentially private dAGMs on decentralized social graphs while preserving the individual privacy of the users. We address three challenges when endeavoring to design PrivAGM.

Challenge 1: How to collect neighbor lists while striking a balance between privacy, utility, and efficiency?

To privately collect neighbor lists, a widely-used method is the randomized neighbor list (RNL) mechanism, which enables users to flip each bit in their neighbor lists with a certain probability to achieve pure edge local DP (LDP) [5]. However, the RNL mechanism significantly disrupts the structure of the decentralized social graph, as it requires flipping a considerable number of bits in each neighbor list to achieve the desired level of privacy protection. Another plausible method is to let users encrypt each bit of their neighbor lists using the MPC technique (e.g., additive secret sharing [19]), enabling the subsequent secure construction of dAGM on the encrypted neighbor lists. However, the construction of dAGM on encrypted neighbor lists incurs significant undesirable performance overhead as the neighbor lists are normally sparse [20]. To strike a balance between privacy, utility, and efficiency, we propose to layer the concept of selective MPC [21] with RNL to encrypt the neighbor lists. Unlike typical secret sharing-based MPC, where all participating parties receive shares for all private values, selective MPC involves randomly selecting a subset of parties to receive shares for each private value. Our selective MPC-based encryption method can achieve pure edge LDP for the neighbor lists *without compromising their utility* or incurring significant undesirable performance overhead.

Challenge 2: How to securely extract the differentially private characteristics essential for dAGM from the collected encrypted social graph while ensuring their utility?

Constructing the differentially private dAGM requires the addition of calibrated noises to the extracted characteristics from the input graph. However, for certain characteristics (e.g., triangle subgraph count), the global sensitivity of the calibrated noises is large, which could severely compromise the utility of the extracted characteristics. To overcome this challenge, we propose the oblivious edge clipping protocol, which enables the oblivious restriction of the maximum degree of the encrypted social graph, thereby mitigating the worst-case impact of a single vertex or edge and achieving a small global sensitivity. Moreover, we develop a suite of MPC-based secure components, including secure degree sequence extraction, secure attribute-edge correlation distribution extraction, secure attribute distribution extraction, and secure triangle counts extraction. Their integration securely captures the differentially private characteristics essential for constructing the dAGM from the encrypted clipped graph.

Challenge 3: How to construct the dAGM to effectively capture the inherent directionality of the edges in the decentralized social graph?

The existing AGMs [3], [7], [10], [18] have mainly focused on undirected graphs, resulting in a lack of capability to capture the inherent directionality of edges within the input graph. This directionality is of utmost importance for various analytics tasks on directed graphs, such as the discovery of social roles [1] and prediction of disease outbreaks [3]. In response, we design a directed version of the widely used Chung-Lu (CL) model [37] to effectively incorporate the edge directionality presented in the degrees of vertices. Building upon the directed CL model, we adapt the classical AGM framework [3] to account for the directionality exhibited in the attribute-edge correlations and triangle subgraph counts. This leads to the dAGM, which generates directed attributed graphs that effectively emulate the inherent edge directionality presented in the input graph.

Evaluation results. We conduct experiments on three real-world social graph datasets. The results clearly demonstrate that, in terms of utility in synthetic graphs, PrivAGM not only outperforms the state-of-the-art method AsgLDP [10] under the decentralized model but also surpasses the state-of-the-art method CAGM [7] under the centralized model. For instance, when the privacy budget is 1 on the Gplus dataset, for the Hellinger distance between the degree distribution of the synthetic and input social graph, PrivAGM is 58.3% lower than AsgLDP, and also is 42.9% lower than CAGM. Additionally, we conduct a study on the privacy budget allocation of PrivAGM and provide guidelines for properly allocating privacy budgets to different characteristics necessary for dAGM construction. We also evaluate the system cost savings using our neighbor list encryption scheme. The results show that with a privacy budget of 1.2,

PrivAGM achieves about 70% savings in the system cost compared to simply encrypting each bit of the neighbor lists.

2. Preliminaries

2.1. Attributed Graphs

This paper focuses on directed and unweighted attributed graphs. An attributed graph is represented as $G = (\mathbf{A}, \mathbf{F})$. Here, $\mathbf{A} \in \{0,1\}^{N \times N}$ represents the (binary) asymmetric adjacency matrix of G with a size of $N \times N$, where N is the number of vertices in G. Each vertex is denoted by $i \in [1, N]$. $\mathbf{A}[i,j] = 1$ indicates the presence of a directed edge (denoted as (i,j)) from vertex i to vertex j, while $\mathbf{A}[i,j] = 0$ indicates the absence of such a directed edge. We exclude self-connected edges in our consideration, meaning that $\mathbf{A}[i,i] = 0, i \in [1, N]$. The number of edges in G is denoted as M. We utilize $\mathbf{A}[i,:]$ (resp. $\mathbf{A}[:,i]$) to denote the i-th row (resp. column) of matrix $\mathbf{A}$, which corresponds to the out-neighbors (resp. in-neighbors) of vertex i. Vertex i's out-neighbors are the vertices reachable by following outgoing edges from vertex i, whereas in-neighbors are the vertices with directed edges pointing towards vertex i. The number of vertex i's out-neighbors (resp. in-neighbors) is named its out-degree (resp. in-degree), denoted as $d_i^+ = \mathsf{sum}(\mathbf{A}[i,:])$ (resp. $d_i^- = \mathsf{sum}(\mathbf{A}[:,i])$). $\mathbf{F} \in \{0,1\}^{N \times L}$ represents the attribute matrix of G with a size of $N \times L$, where L is the dimensions of each vertex's attribute vector and $\mathbf{F}[i,:]$ corresponds to vertex i's attribute vector. The attributes are assumed to be binary, following previous works [3], [7], [10], [18]. However, the concepts presented in the paper can be extended to attributes with larger domains.

We use $\{\sigma_i\}_{i \in [1,\gamma]}$ to denote the set $\{\sigma_1, \cdots, \sigma_\gamma\}$, and omit the subscript $i \in [1, \gamma]$ when it does not impact the presentation. We use Θ to denote a probability distribution, and $\Theta(\upsilon)$ to denote the probability of the value υ.

2.2. Attributed Graph Model (AGM)

The existing AGMs [3], [7], [10], [18] primarily concentrate on undirected graphs rather than the directed graphs addressed in this paper. Next, we provide an overview of the classical and widely adopted AGM [18] as a foundational approach to comprehend the underlying principles of AGM. We will construct our dAGM within the framework established by this approach. The AGM [18] first learns:

- The degree sequence of all vertices: $\mathcal{D} = \{d_i\}_{i \in [1,N]}$.
- The attribute distribution $\Theta_F : \Theta_F(\mathbf{y}) = \frac{n_\mathbf{y}}{N}, \mathbf{y} \in \{0,1\}^L$, where $n_\mathbf{y}$ is the number of vertices in the input graph that have attribute vector equal to $\mathbf{y}$.
- The attribute-edge correlation distribution Θ_X, which is defined as $\Theta_X(\mathbf{y} - \mathbf{y}') = \frac{n_{\mathbf{y}-\mathbf{y}'}}{M}, \mathbf{y} - \mathbf{y}' \in \{0,1\}^L \times \{0,1\}^L$, where $\mathbf{y} - \mathbf{y}'$ is named as *an attribute-edge correlation* (i.e., an edge connecting two vertices with attribute vectors equal to $\mathbf{y}$ and $\mathbf{y}'$, respectively), $n_{\mathbf{y}-\mathbf{y}'}$ is the number of attribute-edge correlation $\mathbf{y} - \mathbf{y}'$ in the input graph.

To generate an attributed graph consisting of N vertices, the AGM [18] first independently samples an attribute vector for each vertex using Θ_F. It then iteratively samples edges based on the new attribute vectors, utilizing $\mathcal{D}$ and Θ_X.

2.3. Differential Privacy for Graphs

Conventional DP [14] ensures that the presence or absence of an individual's data in a dataset does not significantly impact the insights derived from the dataset. Different from conventional DP, which considers the *central model* where a trusted data curator accesses the dataset, LDP [22] considers the *local model*, where each user individually obfuscates their personal data and sends the obfuscated data to a (potentially malicious) data collector. In this paper, we consider two definitions of DP in the context of attributed graphs: edge LDP [5] and edge DP [3].

Edge LDP. Consider $\mathbf{A}[i,:] \in \{0,1\}^N$ as the neighbor list of vertex i, which is named as vertex i's *neighbor list*. Then the definition of edge LDP is as follows [5]:

***Definition* 1.** *A randomized mechanism $\mathcal{M}$ satisfies ε-edge LDP, if and only if for any two neighbor lists $\mathbf{A}[i,:], \mathbf{A}'[i,:]$ that differ in one bit, we have: $\forall \mathbf{b} \in Range(\mathcal{M})$,*

$$Pr[\mathcal{M}(\mathbf{A}[i,:]) = \mathbf{b}] \leq e^\varepsilon \cdot Pr[\mathcal{M}(\mathbf{A}'[i,:]) = \mathbf{b}],$$

where $Range(\mathcal{M})$ denotes the set of all possible outputs of $\mathcal{M}$, ε is the privacy budget.

Edge DP. While edge DP in the context of *unattributed* graphs solely focuses on edge privacy [6], edge DP in the context of *attributed* graphs takes an additional step to safeguard the privacy of vertex attributes. Edge DP in the context of attributed graphs is defined as follows [3]:

***Definition* 2.** *A randomized mechanism $\mathcal{M}$ satisfies (ε, δ)-edge DP, if and only if for any two neighboring attributed graphs G and G' that differ in the presence of a single edge or in the attribute vector associated with a single vertex, we have: $\forall \mathcal{G} \in Range(\mathcal{M})$,*

$$Pr[\mathcal{M}(G) = \mathcal{G}] \leq e^\varepsilon \cdot Pr[\mathcal{M}(G') = \mathcal{G}] + \delta,$$

where ε is the privacy budget and δ is a privacy parameter. If $\delta = 0$, we say that $\mathcal{M}$ provides pure edge DP.

Discrete Laplace distribution. The discrete Laplace distribution $Lap(\cdot)$ is commonly employed to draw discrete noises to provide DP. Its formal definition is as follows [23]:

***Definition* 3.** *A discrete random variable x follows $Lap(\varepsilon, \delta, \Delta)$ if its probability density function is*

$$Pr[x] = \frac{e^{\frac{\varepsilon}{\Delta}} - 1}{e^{\frac{\varepsilon}{\Delta}} + 1} \cdot e^{\frac{-\varepsilon \cdot |x - \mu|}{\Delta}}. \tag{1}$$

where $\mu = -\frac{\Delta}{\varepsilon} \cdot \ln((e^{\frac{\varepsilon}{\Delta}} + 1) \cdot (1 - (1-\delta)^{\frac{1}{\Delta}}))$ is the mean of the distribution and Δ is the sensitivity.

2.4. Additive Secret Sharing

In 2-out-of-2 additive secret sharing (ASS) [19], a secret value $x \in \mathbb{Z}_{2^l}$ is split into two secret shares, $\langle x \rangle_1 \in \mathbb{Z}_{2^l}$ and $\langle x \rangle_2 \in \mathbb{Z}_{2^l}$, which are distributed among parties P_1 and P_2, respectively. We denote the ASS of x as $[\![x]\!]$. If only certain values in an entity Ω (e.g., set or table) are secret-shared, we represent the entity as $[\Omega]$. The basic operations in the ASS domain are as follows: (1) Linear operations: Linear operations on secret-shared values can be computed locally. For example, given constants c and c', and two secret-shared values $[\![x]\!]$ and $[\![y]\!]$, computing

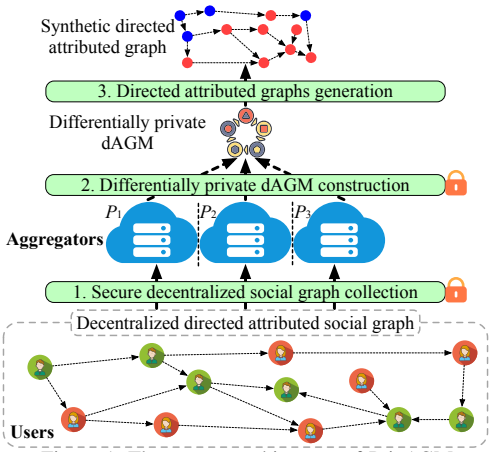


Figure 1. The system architecture of PrivAGM.

$[\![z]\!] = [\![c \cdot x + c' \cdot y]\!]$ involves each party $P_{i\in\{1,2\}}$ locally computing $\langle z\rangle_i = c\cdot\langle x\rangle_i + c'\cdot\langle y\rangle_i$. (2) Multiply operations: Multiply operation on two secret-shared values $[\![x]\!]$ and $[\![y]\!]$ requires one round of online communication. To compute $[\![z]\!] = [\![x \cdot y]\!]$, $P_{1,2}$ utilize a secret-shared Beaver triple $[\![w]\!] = [\![u\cdot v]\!]$ which can be prepared offline. $P_{i\in 1,2}$ first computes $\langle e\rangle_i = \langle x\rangle_i + \langle u\rangle_i$ and $\langle f\rangle_i = \langle y\rangle_i + \langle v\rangle_i$ locally, then communicates with each other to reveal e and f. Finally, $P_{1,2}$ locally compute $\langle z\rangle_1 = e\cdot f + f\cdot\langle u\rangle_1 + e\cdot\langle v\rangle_1 + \langle w\rangle_1$ and $\langle z\rangle_2 = f\cdot\langle u\rangle_2 + e\cdot\langle v\rangle_2 + \langle w\rangle_2$ respectively.

3. Problem Statement

3.1. System Architecture

The system architecture of PrivAGM is illustrated in Fig. 1. Within the system, there are two types of entities: users and aggregators. The N users, along with the connections among them, collectively form the decentralized social graph $G = (\mathbf{A}, \mathbf{F})$. Each user corresponds to a vertex $i \in [1, N]$ and holds a local view $(\mathbf{A}[i,:], \mathbf{F}[i,:])$, where $\mathbf{A}[i,:]$ represents user i's neighbor list held by it (named as *local neighbor list*) and $\mathbf{F}[i,:]$ represents user i's attributes. In contrast to previous studies [5], [10] that primarily focus on idealized decentralized social graphs where each user's local neighbor list *completely* includes its out/in-neighbors, i.e., the adjacency matrix $\mathbf{A}$ is symmetric, our attention is directed towards a more realistic decentralized *directed* social graph scenario. Here, each user's local neighbor list only includes its out-neighbors[1], i.e., $\mathbf{A}$ is asymmetric. Such decentralized social graphs are more commonly encountered in practice (e.g., the phone networks and Diaspora [8]).

The aggregators aim to collect the decentralized social graph data from the users to perform various graph analytics tasks. However, privacy concerns hinder users' participation in the tasks [5], [16]. Our goal is to facilitate secure construction of a dAGM on the decentralized social graph by the aggregators, while ensuring the preservation of individual privacy. In this paper, we leverage a distributed trust framework where three aggregators (referred to as P_1, P_2, and P_3) from different trust domains cooperatively construct the dAGM. Such distributed trust has gained significant interest in academia [24]–[26] and industry [27]–[29].

1. PrivAGM can be effortlessly expanded to the scenario where each user possesses its in-neighbors or out/in-neighbors.

3.2. Threat Model and Security Guarantees

Threat model. We assume that all parties involved, including the aggregators and the users, are honest-but-curious. This means that all parties will adhere to the protocol specification of PrivAGM without deviation, but they may attempt to infer the local views of other users during the execution of PrivAGM. Additionally, we assume that all parties will not collude with each other.

Security guarantees. Under the aforementioned adversary model, PrivAGM guarantees that the adversary is unable to obtain the precise $(\mathbf{A}[i,:], \mathbf{F}[i,:])$ of each user $i \in [1, N]$. More specifically, in the adversary's perspective, PrivAGM guarantees the following: (1) during the decentralized social graph collection phase, the local neighbor lists are differentially private (edge LDP); (2) during the dAGM construction phase, the learned dAGM is differentially private (edge DP). Note that based on the post-processing invariance of DP [14], the synthetic attributed graphs generated by the differentially private dAGM is still differentially private.

3.3. An Overview of PrivAGM

PrivAGM consists of three phases: (1) secure decentralized social graph collection (Section 4), (2) differentially private dAGM construction (Section 5), and (3) directed attributed graphs generation (Section 6). Their workflow is illustrated in Fig. 1. In phase (1), each user carefully encrypts its local view, followed by distributing the ciphertext to the aggregators. In phase (2), the aggregators securely extract the differentially private parameters necessary for dAGM from the encrypted social graph. Lastly, in phase (3), the aggregators utilize the constructed differentially private dAGM to generate synthetic directed attributed graphs.

4. Secure Decentralized Social Graph Collection

We begin by addressing the encryption of each user i's local neighbor list $\mathbf{A}[i,:], i \in [1, N]$. To achieve high efficiency, PrivAGM employs the lightweight ASS technique for encrypting the bits in $\mathbf{A}[i,:]$.

Motivation. To encrypt $\mathbf{A}[i,:]$, a simply method is to directly apply ASS over the entire $\mathbf{A}[i,:]$ of length N (i.e., dense encoding). However, this method is inefficient due to the sparsity of social graphs. According to Facebook's statistics [20], the average user has around 130 friends in a social network, which is much less than the total number of vertices, i.e., N. As a result, $\mathbf{A}[i,:]$ will be mostly filled with zeros, leading to high sparsity. Therefore, applying ASS over the entire $\mathbf{A}[i,:]$ would result in significant uplink communication cost from user i to the aggregators, as well as unnecessary workload during the subsequent dAGM construction on the encrypted social graph. Another simply method is to consider only the non-zero bits in $\mathbf{A}[i,:]$ (i.e., sparse encoding) and apply ASS solely to the indices of these non-zero elements. Specifically, user i first stores $\mathbf{A}[i,:]$ in sparse representation: $E_i = \{(i,j) | \mathbf{A}[i,j] = 1, j \in [1, N]\}$ ($E_i = \{(i,j)\}$ for simplicity), and then encrypts E_i into

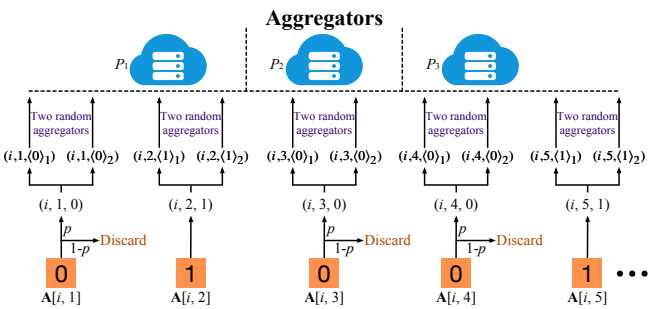


Figure 2. Overview of our local neighbor list encryption algorithm. p refers to the selective probability of each 0 bit in $\mathbf{A}[i,:]$.

$[E_i] = \{(i, [\![j]\!])\}$. However, the resulting ciphertext makes it difficult to securely access the neighbors of each user. Since accessing neighbors is a fundamental operation required to extract parameters necessary for dAGM, the method also imposes a heavy workload on the aggregators.

Observation. We observe that the two aforementioned straightforward methods represent two extremes in local neighbor list encryption: the first one allows efficient accessing of neighbors but sacrifices sparsity, while the second one preserves sparsity but lacks efficient neighbor accessing. The observation indicates that the challenge here lies in preserving the benefits of sparsity while enabling efficient neighbor accessing. Based on the observation, we resort to the concept of edge LDP to protect each local neighbor list. To achieve edge LDP, our starting point is the randomized neighbor list (RNL) mechanism [5]. RNL applies randomized response [30], allowing each user to flip each bit of its local neighbor list with a certain probability. However, RNL significantly disrupts the structure of the decentralized social graph by flipping a considerable number of bits in each local neighbor list in order to achieve the desired level of privacy protection. To address this issue, our insight is to layer the concept of selective MPC [21] with RNL to achieve pure edge LDP for the local neighbor lists *without compromising their utility* or incurring significant undesirable performance overhead. Unlike typical secret sharing-based MPC, where all participating parties receive shares for all private values, selective MPC involves randomly selecting a subset of parties to receive shares for each private value.

Layering selective MPC with RNL. Fig. 2 shows the overview of our proposed local neighbor list encryption algorithm. Given $\mathbf{A}[i,:]$, user i first constructs the set $E_i = \{(i,j,1) | \mathbf{A}[i,j] = 1, j \in [1,N]\}$ to fully capture the nonzero bits in $\mathbf{A}[i,:]$. Next, for every zero bit $\mathbf{A}[i,j] = 0, j \in [1,N]$, user i includes $(i,j,0)$ to E_i *with a probability of* p. The resulting local neighbor list is denoted as $E_i = \{(i,j,b_{ij})\}$, where $b_{ij} = 1$ or 0. User i then applies ASS over each b_{ij} in E_i to obtain the encrypted neighbor list $[E_i] = \{(i,j,[\![b_{ij}]\!])\}$. After that, a straightforward method is to distribute $\{(i,j,\langle b\rangle_1)\}$ and $\{(i,j,\langle b\rangle_2)\}$ to two aggregators, respectively. However, this approach cannot achieve pure edge LDP in the individual view of each aggregator because it solely *adds* zero bits to E_i and does not *remove* nonzero bits from E_i, thereby capturing only additive noise without accounting for subtractive noise. Our insight to address this issue is to leverage the concept of selective MPC [21]. Specifically, for *each* $(i,j,[\![b_{ij}]\!]) \in [E_i]$, user i sends $(i,j,\langle b\rangle_1)$ and $(i,j,\langle b\rangle_2)$ to two *randomly* selected aggregators, respectively. Thus, each aggregator only has access to a random subset of $[E_i]$. In other words, our method simulates removing some nonzero bits from the view of each aggregator, capturing subtractive noise (similar to how adding zero bits captures additive noise), thereby achieving pure edge LDP in each aggregator's view (as proven in Appendix C.2) without compromising the utility.

For each $(i,j,[\![b_{ij}]\!]) \in [E_i]$, user i allows the two selected aggregators to be aware of each other to facilitate subsequent computations in the ASS domain. We use $\{[E_i]^{(1,2)}\}_{i\in[1,N]}$, $\{[E_i]^{(2,3)}\}_{i\in[1,N]}$, and $\{[E_i]^{(3,1)}\}_{i\in[1,N]}$ to represent the encrypted local neighbor lists of all users jointly held by the pairs of aggregators (P_1,P_2), (P_2,P_3), and (P_3,P_1) respectively, and use $\{[E_i]\}_{i\in[1,N]} = \{[E_i]^{(1,2)} \cup [E_i]^{(2,3)} \cup [E_i]^{(3,1)}\}_{i\in[1,N]}$ to represent the encrypted local neighbor lists held collectively by the three aggregators.

Encrypting the attribute vectors. Encrypting the attribute vectors is straightforward. Before encrypting the attribute vector $\mathbf{F}[i,:]$, user i compresses it into a value $f_i \in \mathbb{Z}_{2^L}$ by

$$f_i = \sum\nolimits_{l\in[1,L]} \mathbf{F}[i,l] \cdot 2^l. \tag{2}$$

User i then applies ASS over f_i to produce $[\![f_i]\!]$ and only distributes $\langle f_i\rangle_1$ to P_1 and $\langle f_i\rangle_2$ to P_2.

5. Differentially Private dAGM Construction

We will now describe the process by which the aggregators securely construct a differentially private dAGM on the encrypted social graph they hold. To begin, we introduce the parameters necessary for our designed dAGM, and we will later explain how to securely extract these parameters.

Parameters necessary for dAGM. Our dAGM is built on the widely adopted AGM [18], which involves learning three sets of parameters from the input attributed graph: the degree sequence $\mathcal{D}$, the attribute distribution Θ_F, and the attribute-edge correlation distribution Θ_X. However, the AGM [18] primarily focuses on undirected graphs, whereas PrivAGM specifically addresses directed graphs, where the *inherent directionality* of edges plays a crucial role in accurately mimicking the input graph. Hence, we first extend the AGM [18] by adapting its required parameters, enabling to effectively capture the inherent directionality present in the input directed graph. Since the attribute distribution Θ_F does not possess any directionality, our focus lies solely on adapting the degree sequence $\mathcal{D}$ and the attribute-edge correlation distribution Θ_X to capture the directionality.

When considering the degree sequence, it is important to note that in directed graphs, each vertex has both out-degree and in-degree. Therefore, to effectively capture the directionality using the degree sequence, we specifically tailor our dAGM to learn the sequence of (out-degree, in-degree) pairs, denoted as $\mathcal{D}^{\pm} = \{(d_i^+, d_i^-)\}_{i\in[1,N]}$, where d_i^+ and d_i^- represent the out-degree and in-degree of vertex i, respectively. When considering the attribute-edge correlation distribution, to effectively capture the directionality, we specifically tailor our dAGM to learn $\Theta_{\vec{X}}$: $\Theta_{\vec{X}}(f \rightarrow$

$f') = \frac{n_{f\to f'}}{M}, f\to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$, where $n_{f\to f'}$ represents the number of edges *from* a vertex with attribute f (mapped from an attribute vector by Eq. 2) *to* a vertex with attribute f'. The distinction between our $\Theta_{\vec{X}}$ and the AGM [18]'s Θ_X lies in our consideration of attribute-edge correlations $f\to f'$ and $f'\to f$ as distinct, whereas the AGM [18] treats them as equivalent. As a result, our $\Theta_{\vec{X}}$ effectively captures the directionality inherent in the input directed graph.

Drawing inspiration from [3], the count of triangles in the input graph plays a pivotal role in refining the structure of the synthetic graphs. However, the work [3] considers all triangles in the input graph to be identical, thus failing to capture the directionality of edges. To address this limitation, we propose counting non-isomorphic triangles separately. Specifically, we identify two non-isomorphic triangles (named as "TriA" and "TriB", respectively) in a directed graph (disregarding the vertex attributes):

TriA TriB

Our dAGM tallies the occurrences of TriA, TriB in the input graph as separate counts, denoted as $n^A_\triangle, n^B_\triangle$, respectively.

5.1. Oblivious Edge Clipping

Before delving into the detailed introduction of the parameters extraction, we provide a protocol to enable $P_{1,2,3}$ to obliviously clip the edges in the encrypted social graph.

Motivation of edge clipping. Edge clipping is necessary due to the significant sensitivity Δ involved in achieving edge DP for the attribute-edge correlation distribution $\Theta_{\vec{X}}$ and the triangle counts $n^A_\triangle$ and $n^B_\triangle$. For example, the addition or removal of a single edge changes $n^A_\triangle$ or $n^B_\triangle$ by at most $d_{max} - 1$, where d_{max} is the maximum (out or in)-degree in the input graph, which can range up to $N-1$. Setting such sensitivities and adding noises would completely destroy the utility of the extracted parameters. Hence, to achieve both high utility and edge DP in this scenario, it is crucial to mitigate the worst-case impact of a single vertex or edge. This can be accomplished by applying edge clipping on the input graph to restrict its maximum out/in-degree. A naive approach would be for each user to locally perform edge clipping on their local neighbor lists. However, in the realistic decentralized social graph scenario we are targeting, where each user holds an incomplete neighbor list, it is not feasible for users to effectively restrict the in-degrees by performing edge clipping on local neighbor lists.

Basic oblivious edge clipping. Our oblivious edge clipping algorithm is built on the widely-used edge clipping algorithm in the plaintext domain [31]. It projects an input graph with arbitrary degrees onto a graph where the maximum degree is limited to k. Given a graph and a clipping parameter k, it iterates through each edge of the graph in a random order. If either of the two vertices connected by an edge currently has a degree larger than k, the edge is deleted.

However, the edge clipping algorithm [31] primarily focuses on undirected graphs, which means it does not differentiate between out-degrees and in-degrees. Hence, we adapt the algorithm to accommodate directed graphs. In this context, when provided with a directed graph and a clipping parameter k, our algorithm iterates through each directed edge of the graph in a random order. When considering an edge from vertex i to vertex j, the values of d_i^+, d_i^- are first incremented to 1. After that, if $d_i^+ > k$ or $d_i^- > k$, the edge is deleted. Our algorithm ensures that both the maximum out/in-degree, after edge clipping, are limited to k. We then detail how $P_{1,2,3}$ perform oblivious edge clipping.

Algorithm 1 Basic Oblivious Edge Clipping

Input: Encrypted local neighbor lists $\{[E_i]\}$; clipping parameter k.
Output: Encrypted clipped local neighbor lists $\{[\overline{E}_i]\}$.

▷ $P_{1,2}$ operate on $\{[E_i]^{(1,2)}\}$:
1: Initialization: $[\![d_i^+]\!] = [\![0]\!]; [\![d_i^-]\!] = [\![0]\!]; [\overline{E}_i]^{(1,2)} = \emptyset, i \in [1, N]$.
2: **for all** $(i, j, [\![b_{ij}]\!]) \in [E_i]^{(1,2)}, i \in [1, N]$ **do**
3: $\quad [\![d_i^+]\!] = [\![d_i^+]\!] + [\![b_{ij}]\!]$.
4: $\quad [\![d_j^-]\!] = [\![d_j^-]\!] + [\![b_{ij}]\!]$.
5: $\quad [\![\mathsf{isClip}_1]\!] = \mathsf{secComp}([\![d_i^+]\!], k/3)$.
6: $\quad [\![\mathsf{isClip}_2]\!] = \mathsf{secComp}([\![d_j^-]\!], k/3)$.
7: $\quad [\![\mathsf{isClip}]\!] = [\![\mathsf{isClip}_1]\!] \cdot [\![\mathsf{isClip}_2]\!]$.
8: $\quad [\![b'_{i,j}]\!] = [\![b_{ij}]\!] \cdot [\![\mathsf{isClip}]\!]$. ▷Obliviously clip the edge.
9: $\quad [\overline{E}_i]^{(1,2)}.\mathsf{add}((i, j, [\![b'_{i,j}]\!]))$.
10: **end for**
11: $P_{2,3}$ (resp. $P_{3,1}$) perform above operations on $\{[E_i]^{(2,3)}\}$ (resp. $\{[E_i]^{(3,1)}\}$) to produce $\{[\overline{E}_i]^{(2,3)}\}$ (resp. $\{[\overline{E}_i]^{(3,1)}\}$).
12: **return** $\{[\overline{E}_i]\} = \{[\overline{E}_i]^{(1,2)} \cup [\overline{E}_i]^{(2,3)} \cup [\overline{E}_i]^{(3,1)}\}$.

As stated in Section 4, each aggregator only holds a subset of each local neighbor list, meaning that the (encrypted) edges held by each aggregator are incomplete. Hence, they cannot simply clip the edges using the clipping parameter k. Our main insight is to empower each pair of aggregators, $P_{1,2}$, $P_{2,3}$, and $P_{3,1}$, to *independently* perform oblivious edge clipping on their jointly held encrypted local neighbor lists, i.e., $\{[E_i]^{(1,2)}\}$, $\{[E_i]^{(2,3)}\}$, and $\{[E_i]^{(3,1)}\}$, utilizing a clipping parameter of $k/3$. Due to the uniform random distribution of edges among $P_{1,2,3}$, this method can yield a similar edge clipping outcome as when clipping edges on complete local neighbor lists using parameter k. Algorithm 1 gives our basic oblivious edge clipping protocol.

The process commences as $P_{1,2}$ iterate through each edge $(i, j, [\![b_{ij}]\!]) \in [E_i]^{(1,2)}, i \in [1, N]$, in a random order. Given $(i, j, [\![b_{ij}]\!])$, $P_{1,2}$ increment $[\![b_{ij}]\!]$ to the out-degree $[\![d_i^+]\!]$ of vertex i and the in-degree $[\![d_j^-]\!]$ of vertex j (lines 3 and 4). $P_{1,2}$ then obliviously compare the current $[\![d_i^+]\!]$ and $[\![d_j^-]\!]$ with $k/3$ (lines 5 and 6) using the oblivious comparison protocol $\mathsf{secComp}([\![\alpha]\!], \beta)$, which outputs $[\![1]\!]$ if $\alpha \le \beta$ and $[\![0]\!]$ if $\alpha > \beta$. Here, we employ the well-known and efficient function secret sharing (FSS) based distributed comparison function (DCF) [32] to instantiate $\mathsf{secComp}([\![\alpha]\!], \beta)$. $P_{1,2}$ then multiply the two encrypted comparison results $[\![\mathsf{isClip}_1]\!]$ and $[\![\mathsf{isClip}_2]\!]$ to produce $[\![\mathsf{isClip}]\!]$ (line 7). Here, $\mathsf{isClip} = 1$ signifies that $\mathsf{isClip}_1 = 1$ and $\mathsf{isClip}_2 = 1$, indicating that both the current $d_i^+ \le k/3$ and $d_j^- \le k/3$ hold, and thus the edge $(i, j, [\![b_{ij}]\!])$ does not need to be clipped. $\mathsf{isClip} = 0$ indicates that either $\mathsf{isClip}_1 = 0$ or $\mathsf{isClip}_2 = 0$, meaning that either $d_i^+ > k/3$ or $d_j^- > k/3$, and

Algorithm 2 Efficient Oblivious Edge Clipping

Input: Encrypted local neighbor lists $\{[E_i]\}$; parameters k, w.
Output: Encrypted clipped local neighbor lists $\{[\overline{E}_i]\}$.

//$P_{1,2}$ operate on $\{[E_i]^{(1,2)}\}_{i\in[1,N]}$:
1: Initialization: $[\![d_i^+]\!]=[\![0]\!]$; $[\![d_i^-]\!] = [\![0]\!]$; $[\overline{E}_i]^{(1,2)} = \emptyset, i \in [1, N]$.
▷ Restrict the out-degree of vertex $i\in[1, N]$ **in parallel**:
2: Initialize queue $Q_i = \emptyset$.
3: **for all** $(i, j, [\![b_{ij}]\!]) \in [E_i]^{(1,2)}$ **do**
4: $\quad [\![d_i^+]\!] = [\![d_i^+]\!] + [\![b_{ij}]\!]$.
5: $\quad Q_i.\mathsf{push}((i, j, [\![b_{ij}]\!]))$.
6: $\quad$ **if** $|Q_i| \geq w$ **then**
7: $\quad\quad [\![\mathsf{isClip}]\!] = \mathsf{secComp}([\![d_i^+]\!], k/3)$.
8: $\quad\quad$ **while** $Q_i \neq \emptyset$ **do**
9: $\quad\quad\quad (i, j, [\![b_{ij}]\!]) = Q_i.\mathsf{pop}()$.
10: $\quad\quad\quad [\![b'_{i,j}]\!] = [\![b_{ij}]\!] \cdot [\![\mathsf{isClip}]\!]$. ▷ Obliviously clip the edge.
11: $\quad\quad\quad [\overline{E}_i]^{(1,2)}.\mathsf{add}((i, j, [\![b'_{i,j}]\!]))$.
12: $\quad\quad$ **end while**
13: $\quad$ **end if**
14: **end for**
▷ Restrict the in-degree of vertex $j \in [1, N]$ **in parallel**:
15: Initialize queue $Q_j = \emptyset$.
16: **for all** $(i, j, [\![b'_{ij}]\!]) \in [\overline{E}_i]^{(1,2)}, i \in [1, N]$ **do**
17: $\quad [\![d_j^-]\!] = [\![d_j^-]\!] + [\![b'_{ij}]\!]$.
18: $\quad Q_j.\mathsf{push}((i, j, [\![b'_{ij}]\!]))$; $\quad [\overline{E}_i]^{(1,2)}.\mathsf{remove}((i, j, [\![b'_{ij}]\!]))$.
19: $\quad$ **if** $|Q_j| \geq w$ **then**
20: $\quad\quad [\![\mathsf{isClip}]\!] = \mathsf{secComp}([\![d_j^-]\!], k/3)$.
21: $\quad\quad$ **while** $Q_j \neq \emptyset$ **do**
22: $\quad\quad\quad (i, j, [\![b'_{ij}]\!]) = Q_j.\mathsf{pop}()$.
23: $\quad\quad\quad [\![b''_{i,j}]\!] = [\![b'_{ij}]\!] \cdot [\![\mathsf{isClip}]\!]$. ▷ Obliviously clip the edge.
24: $\quad\quad\quad [\overline{E}_i]^{(1,2)}.\mathsf{add}((i, j, [\![b''_{i,j}]\!]))$.
25: $\quad\quad$ **end while**
26: $\quad$ **end if**
27: **end for**
28: $P_{2,3}$ (resp. $P_{3,1}$) perform above operations on $\{[E_i]^{(2,3)}\}$ (resp. $\{[E_i]^{(3,1)}\}$) to produce $\{[\overline{E}_i]^{(2,3)}\}$ (resp. $\{[\overline{E}_i]^{(3,1)}\}$).
29: **return** $\{[\overline{E}_i]\} = \{[\overline{E}_i]^{(1,2)} \cup [\overline{E}_i]^{(2,3)} \cup [\overline{E}_i]^{(3,1)}\}$.

thus the edge $(i, j, [\![b_{ij}]\!])$ needs to be deleted. Finally, $P_{1,2}$ execute the oblivious edge clipping operation as indicated by line 8. The underlying design concept is that if $\mathsf{isClip} = 0$, b'_{ij} is assigned 0, i.e., the edge is clipped, while if $\mathsf{isClip} = 1$, b'_{ij} is assigned b_{ij}, i.e., the edge is retained. $(i, j, [\![b'_{i,j}]\!])$ is added into the clipped local neighbor list $[\overline{E}_i]^{(1,2)}$.

Likewise, $P_{2,3}$ (resp. $P_{3,1}$) perform above operations on $\{[E_i]^{(2,3)}\}$ (resp. $\{[E_i]^{(3,1)}\}$) to produce $\{[\overline{E}_i]^{(2,3)}\}$ (resp. $\{[\overline{E}_i]^{(3,1)}\}$). Finally, the encrypted clipped local neighbor lists are $\{[\overline{E}_i]\} = \{[\overline{E}_i]^{(1,2)} \cup [\overline{E}_i]^{(2,3)} \cup [\overline{E}_i]^{(3,1)}\}$.

Efficient oblivious edge clipping. Although Algorithm 1 can achieve effective edge clipping, it incurs a significant communication cost. Specifically, as Algorithm 1 iterates through each encrypted edge sequentially, its online communication complexity grows linearly with the number of edges. This results in a complexity of $O(p \cdot N^2)$, where N represents the number of users, each having an encrypted local neighbor list of length $p \cdot N$ in theory. The *sequential* iteration of encrypted edges hinders the possibility of *parallel* processing for them. To address this problem, we propose an efficient oblivious edge clipping protocol based on Algorithm 1, presented in Algorithm 2. This protocol

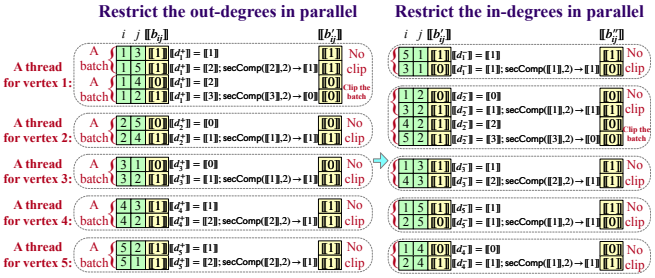


Figure 3. A running example of efficient oblivious edge clipping under the clipping parameter $k = 6$ (i.e., $k/3 = 2$) and batch size $w = 2$.

achieves a favorable online communication round complexity of $O(p \cdot N)$. The enhancement arises from the independence of the out-degree (resp. in-degree) of each vertex from the out-degree (resp. in-degree) of other vertices. Hence, Algorithm 2 enables the aggregators to iterate through each vertex *in parallel*, resulting in a complexity of $O(p \cdot N)$.

Another performance improvement from our proposed *batch edge clipping* technique. We observe that the real edges (i.e., $(i, j, [\![1]\!])$) in the encrypted local neighbor lists are sparse. Hence, to reduce the number of required oblivious comparison operations, we modify the procedure by enabling the aggregators to check whether the current encrypted out-degree $[\![d_i^+]\!]$ or in-degree $[\![d_i^-]\!]$ is larger than $k/3$ after iterating *each batch* of edges, instead of performing the check after iterating each edge. The edges in each batch are then obliviously updated without distinction based on the encrypted comparison result. Given the sparsity of true edges, clipping edges in batch can achieve a similar clipping effect to that of Algorithm 1. We next detail Algorithm 2.

$P_{1,2}$ first restrict the out-degree of each vertex *in parallel* (lines 2-14). For each vertex $i \in [1, N]$, $P_{1,2}$ initialize an empty queue Q_i. Then, while iterating through each edge $(i, j, [\![b_{ij}]\!]) \in [E_i]^{(1,2)}$, they increment vertex i's out-degree $[\![d_i^+]\!]$ by adding $[\![b_{ij}]\!]$ (line 4). Additionally, they push $(i, j, [\![b_{ij}]\!])$ into Q_i (line 5). Unlike Algorithm 1, where a secure comparison (i.e., $\mathsf{secComp}(\cdot)$) is performed after each edge iteration, Algorithm 2 optimizes the cost by performing the secure comparison when the size of Q_i (denoted as $|Q_i|$) exceeds the predefined batch size w. If $|Q_i| \geq w$, $P_{1,2}$ perform line 7 to produce $[\![\mathsf{isClip}]\!]$. The edges within the batch Q_i are obliviously updated *without distinction* based on $[\![\mathsf{isClip}]\!]$ (lines 8-12). After obliviously restricting the out-degrees of all vertices, $P_{1,2}$ proceed to obliviously restrict the in-degree of each vertex *in parallel* on the clipped local neighbor lists $\{[\overline{E}_i]^{(1,2)}\}$ using similar operations (lines 15-27). Similarly, $P_{2,3}$ (resp. $P_{3,1}$) execute the above operations on $\{[E_i]^{(2,3)}\}$ (resp. $\{[E_i]^{(3,1)}\}$) to produce $\{[\overline{E}_i]^{(2,3)}\}$ (resp. $\{[\overline{E}_i]^{(3,1)}\}$).

***Example* 1.** *Fig. 3 illustrates a running example of efficient oblivious edge clipping under $k = 6$ and $w = 2$. In the example, the pair of aggregators first obliviously restrict the out-degrees in parallel (left half of the figure), and then restrict the in-degrees in parallel (right half of the figure).*

5.2. Secure Degree Sequence Extraction

Design rationale. Next, we present the method by which the aggregators securely extract the noisy version of the degree

Algorithm 3 Secure Degree Sequence Extraction

Input: Encrypted clipped local neighbor lists $\{[\overline{E}_i]\}$; maximum out/in-degree k; privacy budget $\varepsilon_{\mathcal{D}}$; privacy parameter $\delta_{\mathcal{D}}$.
Output: Differentially private degree sequence $\widetilde{\mathcal{D}}^{\pm}$.

$\triangleright$ P_1, P_2, and P_3 locally count the encrypted degrees:
1: Initialization: $[\![d_i^+]\!] = [\![0]\!]; [\![d_i^-]\!] = [\![0]\!], i \in [1, N]$.
2: **for all** $(i, j, [\![b_{ij}]\!]) \in [E_i], i \in [1, N]$ **do**
3: $\quad [\![d_i^+]\!] = [\![d_i^+]\!] + [\![b_{ij}]\!]; \quad [\![d_j^-]\!] = [\![d_j^-]\!] + [\![b_{ij}]\!]$.
4: **end for**
5: P_3 secretly shares its shares of degrees with $P_{1,2}$.
6: $P_{1,2}$ add the received shares from P_3 to their own shares to obtain $[\![d_i^+]\!], [\![d_i^-]\!], i \in [1, N]$ in 2-out-of-2 ASS.
7: $P_{1,2}$ arrange the encrypted degrees into table $[\![\mathbf{T}]\!]$, where $[\![\mathbf{T}[i,:]]\!] = [\![d_i^+]\!] \| [\![d_i^-]\!] \| [\![\rho]\!], i \in [1, N]$ and $\rho = 1$.

$\triangleright$ P_3 constructs dummy degree pair table $\mathbf{T}'$:
8: $\mathbf{T}' = []$. $\triangleright$ Empty table.
9: **for all** $(d^+, d^-), d^+ \in [0, k], d^- \in [0, k]$ **do**
10: $\quad n = \mathsf{max}(0, Lap(\varepsilon_{\mathcal{D}}, \delta_{\mathcal{D}}, 4))$.
11: $\quad$ Append n rows of $d^+ \| d^- \| \rho$ to $\mathbf{T}'$, where $\rho = 0$.
12: **end for**
13: P_3 secretly shares $\mathbf{T}'$ with $P_{1,2}$ to produce $[\![\mathbf{T}']\!]$.

$\triangleright$ $P_{1,2}$ perform the remaining operations:
14: Vertically concatenate $[\![\mathbf{T}]\!]$ and $[\![\mathbf{T}']\!]$: $[\![\hat{\mathbf{T}}]\!] = \frac{[\![\mathbf{T}]\!]}{[\![\mathbf{T}']\!]}$.
15: Obliviously shuffle $[\![\hat{\mathbf{T}}]\!]$: $[\![\hat{\mathbf{T}}']\!] = \mathsf{secShuffle}([\![\hat{\mathbf{T}}]\!])$.
16: Safely reveal columns 1 and 2 of $[\![\hat{\mathbf{T}}']\!]$ to obtain $[\hat{\mathbf{T}}']$.
17: Initialize encrypted degree histogram $[\![\mathbf{H}]\!] = [\![0]\!]^{(k+1)\times(k+1)}$.
18: **for all** $d^+ \| d^- \| [\![\rho]\!]$ in $[\hat{\mathbf{T}}']$ **do**
19: $\quad [\![\mathbf{H}[d^+, d^-]]\!] = [\![\mathbf{H}[d^+, d^-]]\!] + [\![\rho]\!]$.
20: **end for**
21: $[\![\widetilde{\mathbf{H}}[d^+, d^-]]\!] = [\![\mathbf{H}[d^+, d^-]]\!] + [\![Lap(\varepsilon_{\mathcal{D}}, 0, 4)]\!], d^+, d^- \in [0, k]$.
22: Safely reveal the differentially private histogram $\widetilde{\mathbf{H}}$.
23: Truncation: $\widetilde{\mathbf{H}}'[d^+, d^-] = \mathsf{max}(0, \widetilde{\mathbf{H}}[d^+, d^-]), d^+, d^- \in [0, k]$.
24: Scale: $\widetilde{\mathbf{H}}''[d^+, d^-] = \widetilde{\mathbf{H}}'[d^+, d^-] \cdot \frac{N}{\mathsf{sum}(\widetilde{\mathbf{H}}')}, d^+, d^- \in [0, k]$.
25: Initialize degree sequence $\widetilde{\mathcal{D}}^{\pm} = \emptyset$.
26: Add $\widetilde{\mathbf{H}}''[d^+, d^-]$ tuples (d^+, d^-) to $\widetilde{\mathcal{D}}^{\pm}$, $d^+, d^- \in [0, k]$.
27: **return** Differentially private degree sequence $\widetilde{\mathcal{D}}^{\pm}$.

sequence $\mathcal{D}^{\pm}$ from the encrypted clipped local neighbor lists $\{[\overline{E}_i]\}$, while ensuring that the extracted noisy sequence (denoted as $\widetilde{\mathcal{D}}^{\pm}$) is differentially private (edge DP). A naive method is to simply add independent secret-shared discrete Laplace noise to the encrypted out/in-degrees, followed by opening the noisy degrees to obtain $\widetilde{\mathcal{D}}^{\pm}$. However, this method introduces a significant amount of noise, particularly in low-degree vertices, which are abundant in real social graphs [3]. To address this issue, our solution is based on the insight that the mapping between a specific vertex and its degree is unimportant for how dAGM uses the degree sequence. What we actually need is the sequence of degrees, and the order is unimportant. Hence, PrivAGM empowers the aggregators to securely extract the differentially private *histogram* of (out-degree, in-degree) pairs, enabling them to derive a more precise $\widetilde{\mathcal{D}}^{\pm}$ from the histogram. Algorithm 3 outlines our secure degree sequence extraction protocol.

Secure degree histogram construction. Initially, the aggregators perform local counting of the encrypted out-degree $[\![d_i^+]\!]$ and in-degree $[\![d_i^-]\!]$ for each vertex $i \in [1, N]$ (lines 1-4). Afterward, the aggregators proceed to securely construct the encrypted histogram $[\![\mathbf{H}]\!]$ with dimensions of $(k+1) \times (k+1)$. The index of each element in $\mathbf{H}$ corresponds to a possible out/in-degree pair $(d^+, d^-), d^+ \in [0, k], d^- \in [0, k]$. The value of $\mathbf{H}(d^+, d^-)$ indicates the count of vertices that possess an out-degree of d^+ and an in-degree of d^-. To construct $[\![\mathbf{H}]\!]$, a straightforward approach involves conducting secure equality tests for each vertex's degree pair, and then add each encrypted test result to the corresponding element of $[\![\mathbf{H}]\!]$. However, the naive method exhibits a prohibitively high complexity of $O((k+1)^2 \cdot N)$. We next present a more efficient method.

We observe that the majority of the cost arises from indexing each encrypted degree pair $([\![d_i^+]\!], [\![d_i^-]\!])$ into $[\![\mathbf{H}]\!]$. If $P_{1,2,3}$ can *safely* reveal these pairs, they can effortlessly index them into $[\![\mathbf{H}]\!]$. To achieve this, our insight is to let $P_{1,2}$ obliviously shuffle the encrypted degree pairs based on a random permutation unknown to them to break the mappings between the encrypted degree pairs and the vertices. However, simply revealing the encrypted degree pairs after the oblivious shuffle will directly expose $\mathbf{H}$. To address this, we let P_3 secretly shares a certain number of dummy entities (flagged as 0) for each possible degree pair with $P_{1,2}$ before the oblivious shuffle. $P_{1,2}$ then perform the oblivious shuffle on the real degree pairs (flagged as 1) together with the dummy degree pairs, followed by reveling all degree pairs (without revealing the flags). The oblivious shuffle guarantees that $P_{1,2}$ cannot discern between the degree pairs that are dummy or real. To ensure the revealed frequency of degree pairs in the views of $P_{1,2}$ is differentially private, the number of dummy entities for each possible degree pair is drawn from a discrete Laplace distribution. However, we cannot directly employ the revealed frequency of degree pairs as $\mathbf{H}$ due to P_3 knowing the number of dummy entities for each degree pair, i.e., the frequency is not differentially private in P_3's view. Instead, PrivAGM lets $P_{1,2}$ add the encrypted flag of each plaintext degree pair to the corresponding element of $[\![\mathbf{H}]\!]$. Next, we detail the process.

P_3 first secretly shares its shares of degree pairs with $P_{1,2}$ to convert the encrypted degree pairs $\{([\![d_i^+]\!], [\![d_i^-]\!])\}$ from the 3-out-of-3 ASS to the 2-out-of-2 ASS (lines 5 and 6). Next, $P_{1,2}$ arrange the encrypted degrees into a table $[\![\mathbf{T}]\!]$ of size $N \times 3$. Here, each row $[\![\mathbf{T}[i,:]]\!] = [\![d_i^+]\!] \| [\![d_i^-]\!] \| [\![\rho]\!], i \in [1, N]$, where $\rho = 1$ is the flag indicating that the degree pairs are real and "$\|$" denotes the concatenation of values into a vector. P_3 then constructs a dummy degree pair table $\mathbf{T}'$ (lines 8-12). Specifically, for each possible degree pair $(d^+, d^-), d^+, d^- \in [0, k]$, P_3 draws a noise n from the discrete Laplace distribution $Lap(\varepsilon_{\mathcal{D}}, \delta_{\mathcal{D}}, 4)$ and truncates n to 0, i.e., $n = \mathsf{max}(0, Lap(\varepsilon_{\mathcal{D}}, \delta_{\mathcal{D}}, 4))$. Here, the sensitivity Δ is set to 4, as the addition or removal of an edge changes the out/in-degrees of exactly two vertices, resulting in changes to the frequency of at most four degree pairs. P_3 then appends n rows of $d^+ \| d^- \| \rho$ to $\mathbf{T}'$, where ρ is set to 0 to indicate that the degree pairs are dummy. Once $\mathbf{T}'$ is constructed, P_3 secretly shares it with $P_{1,2}$ to obtain $[\![\mathbf{T}']\!]$. $P_{1,2}$ proceed by vertically concatenating $[\![\mathbf{T}]\!]$ and

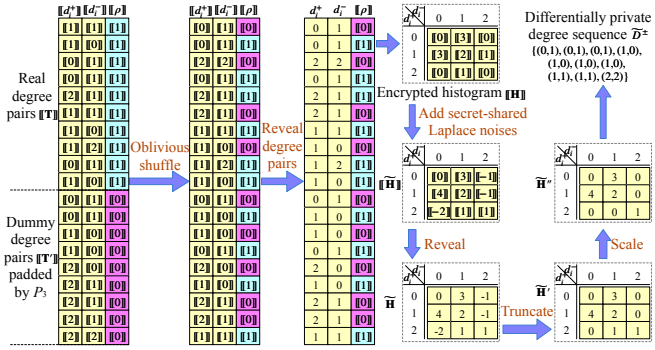


Figure 4. A running example of secure degree sequence extraction under the number of vertices $N = 10$ and the maximum degree $k = 2$.

$[\![\mathbf{T}']\!]$: $[\![\hat{\mathbf{T}}]\!] = \frac{[\![\mathbf{T}]\!]}{[\![\mathbf{T}']\!]}$. They then obliviously shuffle $[\![\hat{\mathbf{T}}]\!]$ by rows: $[\![\hat{\mathbf{T}}']\!] = \mathsf{secShuffle}([\![\hat{\mathbf{T}}]\!])$. Here, we utilize the efficient oblivious shuffle protocol from [33]. Afterwards, $P_{1,2}$ safely reveal columns 1 and 2 (i.e., the out/in-degree pairs) of $[\![\hat{\mathbf{T}}']\!]$ to obtain $[\hat{\mathbf{T}}']$, where each row is in the form of $d^+ \| d^- \| [\![\rho]\!]$. It will be proven in Appendix C.2 that the revealed $[\hat{\mathbf{T}}']$ is differentially private in the individual views of $P_{1,2}$, even if the noises $\{n\}$ drawn by P_3 may be truncated to 0.

Finally, $P_{1,2}$ locally construct $[\![\mathbf{H}]\!]$ based on $[\hat{\mathbf{T}}']$ (lines 17-20). Specifically, since $P_{1,2}$ have the plaintext columns 1 and 2 of $[\hat{\mathbf{T}}']$, which correspond to the out-degrees and in-degrees, respectively, they can easily accumulate $[\![\rho]\!]$ to the corresponding element of $[\![\mathbf{H}]\!]$ by $[\![\mathbf{H}[d^+, d^-]]\!] = [\![\mathbf{H}[d^+, d^-]]\!] + [\![\rho]\!]$ for each row $d^+ \| d^- \| [\![\rho]\!]$ of $[\hat{\mathbf{T}}']$. Since the real degree pairs have $\rho = 1$ but the dummy degree pairs have $\rho = 0$, $[\![\mathbf{H}]\!]$ accurately represents the degree histogram.

Secure degree sequence construction. Then, $P_{1,2}$ proceed to construct $\widetilde{\mathcal{D}}^{\pm}$ based on $[\![\mathbf{H}]\!]$ (lines 21-26). $P_{1,2}$ first locally add a secret-shared noise drawn from $Lap(\varepsilon_{\mathcal{D}}, 0, 4)$ to each element of $[\![\mathbf{H}]\!]$: $[\![\widetilde{\mathbf{H}}[d^+, d^-]]\!] = [\![\mathbf{H}[d^+, d^-]]\!] + [\![Lap(\varepsilon_{\mathcal{D}}, 0, 4)]\!], d^+, d^- \in [0, k]$. The secret-shared noises can be generated offline by following the protocol described in Section 4 of [34]. $P_{1,2}$ then safely reveal the differentially private histogram $[\![\widetilde{\mathbf{H}}]\!]$ to obtain $\widetilde{\mathbf{H}}$, and construct $\widetilde{\mathcal{D}}^{\pm}$ based on $\widetilde{\mathbf{H}}$. Specifically, $P_{1,2}$ first truncate $\widetilde{\mathbf{H}}$ to 0: $\widetilde{\mathbf{H}}'[d^+, d^-] = \mathsf{max}(0, \widetilde{\mathbf{H}}[d^+, d^-]), d^+, d^- \in [0, k]$. Then, $\widetilde{\mathbf{H}}'$ is scaled to ensure that the sum of the histogram is equal to N: $\widetilde{\mathbf{H}}''[d^+, d^-] = \widetilde{\mathbf{H}}'[d^+, d^-] \cdot \frac{N}{\mathsf{sum}(\widetilde{\mathbf{H}}')}, d^+, d^- \in [0, k]$. Finally, for each possible degree pair $(d^+, d^-), d^+, d^- \in [0, k]$, $P_{1,2}$ add $\widetilde{\mathbf{H}}''[d^+, d^-]$ tuples (d^+, d^-) to $\widetilde{\mathcal{D}}^{\pm}$ to construct $\widetilde{\mathcal{D}}^{\pm}$. It will be proven in Appendix C.2 that $\widetilde{\mathcal{D}}^{\pm}$ is differentially private (edge DP) in the individual views of P_1, P_2, and P_3.

***Example* 2.** *Fig. 4 shows a running example of Algorithm 3.*

5.3. Secure Attribute-Edge Correlation Distribution Extraction

We now present how $P_{1,2,3}$ securely extract the noisy version of attribute-edge correlation distribution $\Theta_{\vec{X}}$ from $\{[\overline{E}_i]\}$ and encrypted attributes $\{[\![f_i]\!]\}$, while ensuring that the extracted noisy distribution (denoted as $\widetilde{\Theta}_{\vec{X}}$) is differentially private. Similar to the secure extraction of degree sequence, $P_{1,2,3}$ first securely extract the differentially private *histogram* of attribute-edge correlations, and then derive $\widetilde{\Theta}_{\vec{X}}$ from the histogram. Due to space limitations, we detail the protocol in the full version [35].

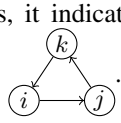
5.4. Secure Attribute Distribution Extraction

We then present how $P_{1,2,3}$ securely extract the noisy version of attribute distribution Θ_F from $\{[\![f_i]\!]\}$, while ensuring that the extracted noisy distribution (denoted as $\widetilde{\Theta}_F$) is differentially private. Similarly, $P_{1,2,3}$ first securely extract the differentially private histogram of attributes from $\{[\![f_i]\!]\}$, then derive $\widetilde{\Theta}_F$ from the histogram. Due to space limitations, we detail the protocol in the full version [35].

5.5. Secure Triangle Counts Extraction

Design rationale. We then present the method by which the aggregators securely extract the noisy version of triangle counts $n_{\triangle}^A$ and $n_{\triangle}^B$ on $\{[\overline{E}_i]\}$, while ensuring that the extracted noisy counts (denoted as $\widetilde{n}_{\triangle}^A$ and $\widetilde{n}_{\triangle}^B$) are differentially private. The underlying idea is to allow the aggregators to iterate over each edge in $\{[\overline{E}_i]\}$ that are connected in pairs and increment the encrypted product of flags of the three edges to the corresponding encrypted triangle count. For example, to securely count TriA, the aggregators iterate over each edge that is connected end-to-end, i.e., $(i, j, [\![b_{ij}]\!]) \in [E_i], (j, k, [\![b_{jk}]\!]) \in [E_j], (k, i, [\![b_{ki}]\!]) \in [E_k]$, and perform $[\![n_{\triangle}^A]\!] = [\![n_{\triangle}^A]\!] + [\![b_{ij}]\!] \cdot [\![b_{jk}]\!] \cdot [\![b_{ki}]\!]$. The idea is correct since only when $b_{ij} = 1$, $b_{jk} = 1$, and $b_{ki} = 1$, which indicates that the three edges are real and indeed form a triangle, $[\![n_{\triangle}^A]\!]$ will be incremented by 1.

However, since secure multiplication in the secret sharing domain necessitates communication between the two involved parties, the aforementioned method can only count triangles whose three edges are held by the *same* pair of aggregators. To address this issue, our solution is based on the observation: the probability of the three edges of each triangle being distributed to the same pair of aggregators is $3 \cdot \frac{1}{3} \cdot \frac{1}{3} \cdot \frac{1}{3} = \frac{1}{9}$. Therefore, PrivAGM enables each pair of aggregators to securely count triangles over the encrypted edges they jointly hold. The encrypted triangle counts are then multiplied by 9 to obtain approximate $[\![n_{\triangle}^A]\!]$ and $[\![n_{\triangle}^B]\!]$. Finally, the differentially private triangle counts $\widetilde{n}_{\triangle}^A$ and $\widetilde{n}_{\triangle}^B$ are safely revealed. Algorithm 4 outlines the protocol.

Design details. Initially, $P_{1,2,3}$ initialize the encrypted counts of TriA and TriB as $[\![n_{\triangle}^A]\!] = [\![0]\!]$ and $[\![n_{\triangle}^B]\!] = [\![0]\!]$ (line 1). Note that, for ease of presentation, $P_{1,2,3}$ initialize $[\![n_{\triangle}^A]\!], [\![n_{\triangle}^B]\!]$ as 3-out-of-3 ASS. Afterwards, $P_{1,2}$ securely count TriA using $\{[\overline{E}_i]^{(1,2)}\}$ they jointly possess (lines 2-10). Specifically, $P_{1,2}$ iterate each *possible* TriA on $\{[\overline{E}_i]^{(1,2)}\}$ by a three-layer loop: for each vertex $i : i \in [1, N]$, iterate vertex i's each (possible) out-neighbor $j : j < i$ and $(i, j, [\![b_{ij}]\!]) \in [\overline{E}_i]^{(1,2)}$, and then iterate vertex j's each (possible) out-neighbor $k : k < i$ and $(j, k, [\![b_{jk}]\!]) \in [\overline{E}_j]^{(1,2)}$. Here, we impose the condition $j < i$ and $k < i$ to prevent double counting. If $(k, i, [\![b_{ki}]\!]) \in [\overline{E}_k]^{(1,2)}$ holds, it indicates a *possible* TriA formed by vertices i, j, k, i.e., 

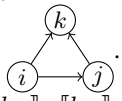
. $P_{1,2}$ then calculate

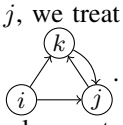
Algorithm 4 Secure Triangle Counts Extraction

Input: Encrypted clipped local neighbor lists $\{[\overline{E}_i]\}$; privacy budget ε_T; maximum out/in-degree k.
Output: Differentially private triangle counts $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$.

1: Initialize $[\![n^A_\triangle]\!] = [\![0]\!]$; $[\![n^B_\triangle]\!] = [\![0]\!]$ in 3-out-of-3 ASS.
▷ $P_{1,2}$ operate on $\{[\overline{E}_i]^{(1,2)}\}$:
▷ Securely count TriA:
2: **for all** $i : i \in [1, N]$ **in parallel do**
3: **for all** $j : j < i$ and $(i, j, [\![b_{ij}]\!]) \in [\overline{E}_i]^{(1,2)}$ **do**
4: **for all** $k : k < i$ and $(j, k, [\![b_{jk}]\!]) \in [\overline{E}_j]^{(1,2)}$ **do**
5: **if** $(k, i, [\![b_{ki}]\!]) \in [\overline{E}_k]^{(1,2)}$ **then**
6: $[\![n^A_\triangle]\!] = [\![n^A_\triangle]\!] + [\![b_{ij}]\!] \cdot [\![b_{jk}]\!] \cdot [\![b_{ki}]\!]$.
7: **end if**
8: **end for**
9: **end for**
10: **end for**
▷ Securely count TriB:
11: **for all** $i : i \in [1, N]$ **in parallel do**
12: **for all** $j, k : j \neq k$ and $(i, j, [\![b_{ij}]\!]), (i, k, [\![b_{ik}]\!]) \in [\overline{E}_i]^{(1,2)}$ **do**
13: **if** $(j, k, [\![b_{jk}]\!]) \in [\overline{E}_j]^{(1,2)}$ **then**
14: $[\![n^B_\triangle]\!] = [\![n^B_\triangle]\!] + [\![b_{ij}]\!] \cdot [\![b_{ik}]\!] \cdot [\![b_{jk}]\!]$.
15: **end if**
16: **end for**
17: **end for**
18: $P_{2,3}$ (resp. $P_{3,1}$) securely count triangles on $\{[\overline{E}_i]^{(2,3)}\}$ (resp. $\{[\overline{E}_i]^{(3,1)}\}$) as described above and aggregate the encrypted triangle counts to $[\![n^A_\triangle]\!]$ and $[\![n^B_\triangle]\!]$.
19: $[\![n^A_\triangle]\!] = 9 \cdot [\![n^A_\triangle]\!]$; $[\![n^B_\triangle]\!] = 9 \cdot [\![n^B_\triangle]\!]$.
20: $[\![\widetilde{n}^A_\triangle]\!] = [\![n^A_\triangle]\!] + [\![Lap(\varepsilon_T, 0, k-1)]\!]$.
21: $[\![\widetilde{n}^B_\triangle]\!] = [\![n^B_\triangle]\!] + [\![Lap(\varepsilon_T, 0, k-1)]\!]$.
22: Safely reveal the differentially private counts $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$.
23: **return** Differentially private triangle counts $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$.

$[\![n^A_\triangle]\!] = [\![n^A_\triangle]\!] + [\![b_{ij}]\!] \cdot [\![b_{jk}]\!] \cdot [\![b_{ki}]\!]$ (line 6). Note that $[\![n^A_\triangle]\!]$ is in 3-out-of-3 ASS, but the product $[\![b_{ij}]\!] \cdot [\![b_{jk}]\!] \cdot [\![b_{ki}]\!]$ is in 2-out-of-2 ASS. Hence, $P_{1,2}$ individually add their respective secret shares of the encrypted product to $\langle n^A_\triangle \rangle_1$ and $\langle n^A_\triangle \rangle_2$, while $\langle n^A_\triangle \rangle_3$ held by P_3 remains unchanged.

$P_{1,2}$ then securely count TriB on $\{[\overline{E}_i]^{(1,2)}\}$ (line 11). Specifically, $P_{1,2}$ iterate each *possible* TriB on $[\overline{E}_i]^{(1,2)}$ by a three-layer loop: for each vertex $i : i \in [1, N]$, iterate vertex i's each pair of (possible) out-neighbors $j, k : j \neq k$ and $(i, j, [\![b_{ij}]\!]), (i, k, [\![b_{ik}]\!]) \in [\overline{E}_i]^{(1,2)}$. If $(j, k, [\![b_{jk}]\!]) \in [\overline{E}_j]^{(1,2)}$ holds, it indicates a *possible* TriB formed by vertices i, j, k, i.e., 

. $P_{1,2}$ then proceed to calculate $[\![n^B_\triangle]\!] = [\![n^B_\triangle]\!] + [\![b_{ij}]\!] \cdot [\![b_{ik}]\!] \cdot [\![b_{jk}]\!]$ (line 14). If there is a bidirectional edge between vertices k, j, we treat the vertices i, j, k as forming two triangles, i.e., 

.

$P_{2,3}$ (resp. $P_{3,1}$) calculate the encrypted triangle counts on $\{[\overline{E}_i]^{(2,3)}\}$ (resp. $\{[\overline{E}_i]^{(3,1)}\}$) as described above and aggregate the encrypted counts to $[\![n^A_\triangle]\!]$ and $[\![n^B_\triangle]\!]$. Subsequently, the aggregators locally multiply $[\![n^A_\triangle]\!]$ and $[\![n^B_\triangle]\!]$ by 9. Next, the aggregators securely construct the differentially private triangle counts by adding secret-shared noises drawn from $Lap(\varepsilon_T, 0, k-1)$ to $[\![n^A_\triangle]\!]$ and $[\![n^B_\triangle]\!]$, resulting in the differentially private triangle counts $[\![\widetilde{n}^A_\triangle]\!]$ and $[\![\widetilde{n}^B_\triangle]\!]$. Here, the sensitivity is set to $k-1$ since adding or removing a single edge can change the triangle count $n^A_\triangle$ or $n^B_\triangle$ by at most $k-1$. Finally, the aggregators safely reveal $\widetilde{n}^A_\triangle, \widetilde{n}^B_\triangle$.

Remark. We acknowledge that performing Algorithm 4 sequentially would entail a significant number of communication rounds. However, as the iterations in the two outermost loops (i.e., lines 2 and 11) are independent of each other, we can let the aggregators execute them in parallel. Moreover, we observe that the primary cost arises from the secure 3-input multiplication gates in lines 6 and 14. To speed up Algorithm 4, we use the technique outlined in Section B.3.1 of [36] to reduce the online communication cost of each secure 3-input multiplication gate from 2 rounds and 8 elements to 1 round and 6 elements. Additionally, we can employ a random sampling strategy to further reduce the cost. Specifically, for the two outermost loops, instead of iterating over each vertex, we can only iterate over η randomly sampled vertices, and then multiply the encrypted triangle counts by $\frac{N}{\eta}$ to obtain the estimated triangle counts.

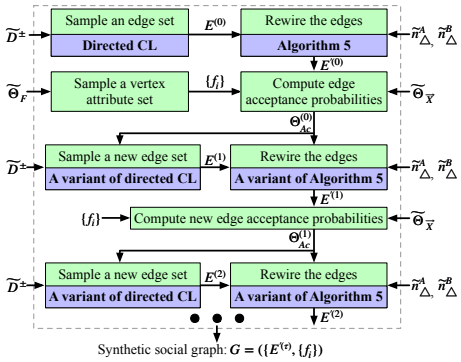


Figure 5. Overview of directed attributed graphs generation.

6. Directed Attributed Graphs Generation

In this section, we introduce how the extracted dAGM parameters can be effectively utilized to generate directed attributed graphs. Note that in this section, we do not refer back to the raw input social graph. Hence, privacy protection measures for the input social graph are not taken into consideration. Our algorithm for generating directed attributed graphs is built upon the classical AGM framework [3]. However, as previously mentioned, the AGM [3] primarily focuses on undirected graphs. Hence, our primary contribution in this section is centered around the adaptation and customization of the AGM framework [3] to effectively cater for the requirements of generating directed graphs.

Overview. Fig. 5 provides an overview of our algorithm for generating directed attributed graphs. The process begins by utilizing a directed version of the Chung-Lu (CL) model [37] to sample an initial (directed) edge set $E^{(0)} = \{(i, j)\}$ based on the degree sequence $\widetilde{\mathcal{D}}^{\pm}$ extracted from the input social graph. Next, Algorithm 5 is employed to rewire the edges to ensure that the counts of TriA and TriB formed by $E^{(0)}$ are no less than $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$, respectively. The rewired edges are denoted as $E'^{(0)}$. At the same time, the vertex attribute set $\{f_i\}_{i \in [1,N]}$ (where f_i for vertex $i, i \in [1, N]$) is sampled

from the attribute distribution $\widetilde{\Theta}_F$. The accept/reject sampling technique is then applied to generate new edge sets. Firstly, the edge acceptance probabilities $\Theta_{Ac}^{(0)}$ are computed based on the current generated graph $(E'^{(0)}, \{f_i\})$ and the attribute-edge correlation distribution $\widetilde{\Theta}_{\vec{X}}$ extracted from the input social graph. Then a new edge set $E^{(1)}$ is sampled through the directed CL model and a variant of Algorithm 5 considering $\Theta_{Ac}^{(0)}$. This process iterates until Θ_{Ac} converge. We then begin by introducing our proposed directed CL model, and then delve into the design intuition of Algorithm 5, and finally, we bring all the design elements together.

6.1. Directed CL Model

The CL model [37] is one of the most popular generative (unattributed and undirected) graph models. In this model, a graph is generated through two key steps. Firstly, each vertex is assigned a desired degree based on the degree sequence $\{d_i\}_{i\in[1,N]}$ of the input graph. Secondly, $\sum_{i\in[1,N]} d_i$ edges are sampled using a probability that is proportional to the desired degrees of the endpoint vertices. Specifically, an edge between vertices i, j is sampled with a probability of $\frac{d_i \cdot d_j}{\sum_{i\neq j, i,j\in[1,N]} d_i \cdot d_j}$. However, we observe that the CL model fails to capture the inherent directionality of edges present in the input *directed* graph. To overcome this limitation, we propose the *directed CL model* (denoted as $\mathsf{directedCL}(\cdot)$).

In our directed CL model , we first assign each vertex $i \in [1, N]$ a desired out/in-degree pair (d_i^+, d_i^-) based on $\widetilde{\mathcal{D}}^{\pm}$. Then, for each directed edge from vertex i to vertex j, we sample it with a probability proportional to the out-degree of vertex i and the in-degree of vertex j, i.e., $\frac{d_i^+ \cdot d_j^-}{\sum_{i\neq j, i,j\in[1,N]} d_i^+ \cdot d_j^-}$. If the edge (i, j) is selected, (i, j) will be included in the initial edge set $E^{(0)}$. To maintain the expected degree distribution, we sample $\max(\sum_{i\in[1,N]} d_i^+, \sum_{i\in[1,N]} d_i^-)$ edges. Clearly, our directed CL model successfully captures the inherent directionality of edges in the input graph by sampling each edge based on the out-degree of the edge's starting vertex and the in-degree of the edge's ending vertex.

6.2. Refining Graph Structure with Edge Rewiring

The graphs generated by the directed CL model only capture the degree information of the input graph, which is insufficient to simulate the structure of the input graph. To address this limitation, the AGM [3] proposes an iterative edge rewiring approach to ensure that the number of triangles in the generated graph is at least equal to that of the input graph, thereby enhancing the structural representation of the generated graph. However, when considering directed graphs, the AGM [3] overlooks the inherent directionality of edges in the input graph and treats all triangles as identical. To overcome this limitation in the context of directed graphs, we propose Algorithm 5 (denoted as $\mathsf{Rewire}(\cdot)$). We detail Algorithm 5 in Appendix A.

6.3. Putting It All Together

We now present the end-to-end workflow for synthesizing directed graphs. We detail the protocol in Appendix B.

TABLE 1. DATASET STATISTICS

Dataset	Type	Vertices	Edges	Density
MOOC	Directed	7047	411749	Dense
Twitter	Directed	81306	1768149	Sparse
Gplus	Directed	107614	13673453	Dense

7. Privacy and Security Analysis

We now present a comprehensive analysis of the privacy and security guarantees provided by PrivAGM in the secure construction of differentially private dAGM on decentralized social graphs. We detail the analysis in Appendix C. At a high-level, in Appendix C.1, we parametrize the leakage function $\mathcal{L}$, which delineates the additional knowledge that the adversary may acquire during the construction process of dAGM. In Appendix C.2, we conduct a formal analysis to prove that the output of $\mathcal{L}$ is differentially private. In Appendix C.3, we undertake a formal analysis to demonstrate that the adversary's view during the construction process of dAGM can be simulated solely based on the output of $\mathcal{L}$.

8. Performance Evaluation

8.1. Setup

We develop a prototype implementation of PrivAGM using a combination of Python and C++. All experiments are conducted on a workstation equipped with 24 Intel Xeon Gold 6240R CPU cores, a NVIDIA RTX A6000 GPU, 128 GB of RAM, and 2 TB of external SSD storage, running Ubuntu 20.04.3 LTS. Additionally, we use a MacBook Air with 8 GB of RAM to simulate users, encrypt the local views, and distribute them to the workstation. Our experiments consider a WAN setting with a maximum throughput of 40MB/s and an RTT latency of 40 ms. All public and private values are encoded in $\mathbb{Z}_{2^{32}}$. We set the privacy parameters $\delta_{\mathcal{D}} = \delta_X = \delta_F = 10^{-5}$. The clipping parameter k follows the previous work [3] and is set to $\sqrt[3]{N}$. The clipping batch size w is set to $\sqrt{N}$, and the sampling number η in the two outermost loops of Algorithm 4 is set to $\sqrt{N}$.

Social graph datasets. We utilize three real-world directed social graph datasets: MOOC[2], Twitter[3], and Gplus[4]. Table 1 presents the statistics of these datasets. For all datasets, we consider ten attributes, which is significantly more than the previous works [3], [18], who only consider two attributes.

Baselines. We consider two state-of-the-art solutions, CAGM [7] and AsgLDP [10], as the baselines for utility comparison in attributed graph synthesis. Originally designed for *undirected* graphs, we modify the baselines to suit our directed social graph datasets by disregarding edge directionality in the datasets and introducing random directions to the edges of the synthetic graphs they generate. As AsgLDP does not converge when generating the synthetic graphs, we select the best synthetic graphs as its results.

Utility metrics. We evaluate the utility of the synthetic graphs using four widely-used metrics [3], [5], [7], [10].

2. https://snap.stanford.edu/data/act-mooc.html
3. https://snap.stanford.edu/data/ego-Twitter.html
4. https://snap.stanford.edu/data/ego-Gplus.html

TABLE 2. $\mathsf{Overall}_{\mathsf{diff}}$ BETWEEN THE INPUT AND THE SYNTHETIC GRAPHS WITH DIFFERENT PRIVACY BUDGET ALLOCATIONS

$[\varepsilon_{\mathcal{D}}, \varepsilon_X, \varepsilon_F, \varepsilon_\triangle]$	MOOC	Twitter	Gplus
$[0.3, 0.1, 0.2, 0.2]$	0.191	0.185	0.221
$[0.3, 0.1, 0.4, 0.1]$	0.184	0.177	0.195
$[0.3, 0.2, 0.1, 0.2]$	0.177	0.163	0.181
$[0.3, 0.3, 0.2, 0.1]$	0.171	0.159	0.194
$[0.3, 0.4, 0.1, 0.1]$	0.174	0.151	0.189
$[0.4, 0.1, 0.1, 0.2]$	0.163	0.132	0.154
$[0.4, 0.2, 0.2, 0.1]$	0.166	0.137	0.157
$[0.4, 0.3, 0.1, 0.1]$	0.159	0.135	0.155
$[0.5, 0.1, 0.2, 0.1]$	0.134	0.125	0.139
$[0.5, 0.2, 0.1, 0.1]$	0.126	0.119	0.125
$[0.6, 0.1, 0.1, 0.1]$	**0.108**	**0.096**	**0.113**

• *Degree distribution:* We utilize the Kolmogorov-Smirnov statistic to evaluate how well a synthetic graph captures the degree distribution of the input graph (denoted as $\mathsf{KS}_{\mathsf{degree}}$), which quantifies the maximum distance between two distributions. Let $c_{\mathcal{D}^\pm}$ and $c'_{\mathcal{D}^\pm}$ represent the cumulative distribution functions estimated from the sorted degree pair sequences of the input and synthetic graphs, respectively. Then $\mathsf{KS}_{\mathsf{degree}} = \max_{d^+=0}^{d^+_{max}} \max_{d^-=0}^{d^-_{max}} |c_{\mathcal{D}^\pm}(d^+, d^-) - c'_{\mathcal{D}^\pm}(d^+, d^-)|$, where d^+_{max}/d^-_{max} are the maximum out/in-degree in the input and synthetic graphs. $c_{\mathcal{D}^\pm}(d^+, d^-)$ (and $c'_{\mathcal{D}^\pm}(d^+, d^-)$) represents the percentage of vertices in the input (and synthetic) graph whose out-degrees $\leq d^+$ and in-degrees $\leq d^-$. As $\mathsf{KS}_{\mathsf{degree}}$ is less sensitive to differences in the tails of the degree distributions, we additionally report the Hellinger distance (denoted as $\mathsf{HD}_{\mathsf{degree}}$) between the two degree distributions $\Theta_{\mathcal{D}^\pm}$ and $\Theta'_{\mathcal{D}^\pm}$: $\mathsf{HD}_{\mathsf{degree}} = \frac{1}{\sqrt{2}} \cdot \sqrt{\sum_{d^+=0}^{d^+_{max}} \sum_{d^-=0}^{d^-_{max}} (\sqrt{\Theta_{\mathcal{D}^\pm}(d^+, d^-)} - \sqrt{\Theta'_{\mathcal{D}^\pm}(d^+, d^-)})^2}$.
• *Attribute-edge correlation distribution:* To evaluate how well a synthetic graph captures the attribute-edge correlations of the input graph, we report the mean relative error (MRE) between their attribute-edge correlation distributions (denoted as $\mathsf{MRE}_{\mathsf{corr}}$), and the Hellinger distance: $\mathsf{HD}_{\mathsf{corr}} = \frac{1}{\sqrt{2}} \cdot \sqrt{\sum_{f=0}^{2^L} \sum_{f'=0}^{2^L} (\sqrt{\Theta_{\vec{X}}(f \to f')} - \sqrt{\Theta'_{\vec{X}}(f \to f')})^2}$, where $\Theta_{\vec{X}}(f \to f')$ is the proportion of edges that connect a vertex with attribute value f to a vertex with attribute value f'.
• *Local clustering coefficient:* The local clustering coefficient C_i for a vertex i is determined by the proportion of edges between the vertices within its neighborhood divided by the number of edges that could potentially exist between them. In directed graphs, $C_i = \frac{|\{(j,k): j,k \in Ne_i, (j,k) \in E\}|}{(d_i^+ + d_i^-) \cdot (d_i^+ + d_i^- - 1)}$, where Ne_i is the set of vertex i's out/in-neighbors, and E is the edge set [38]. The average of the local clustering coefficient is $C = \frac{\sum_{i \in [1,N]} C_i}{N}$. We report the MRE between C of the synthetic and input graphs (denoted as $\mathsf{MRE}_{\mathsf{cluster}}$).
• *Triangle counts:* We report the MRE in the number of triangles, TriA and TriB, between the synthetic and input graphs (denoted as $\mathsf{MRE}_{\mathsf{TriA}}$ and $\mathsf{MRE}_{\mathsf{TriB}}$ respectively).

8.2. Privacy Budget Allocation

We first determine which is the best privacy budget allocation of PrivAGM by empirically measure the overall difference $\mathsf{Overall}_{\mathsf{diff}}$ on different datasets. $\mathsf{Overall}_{\mathsf{diff}}$ is calculated as the average of the above metrics: $\mathsf{Overall}_{\mathsf{diff}} = \frac{\mathsf{KS}_{\mathsf{degree}} + \mathsf{HD}_{\mathsf{degree}} + \mathsf{MRE}_{\mathsf{corr}} + \mathsf{HD}_{\mathsf{corr}} + \mathsf{MRE}_{\mathsf{cluster}} + \mathsf{MRE}_{\mathsf{TriA}} + \mathsf{MRE}_{\mathsf{TriB}}}{7}$.

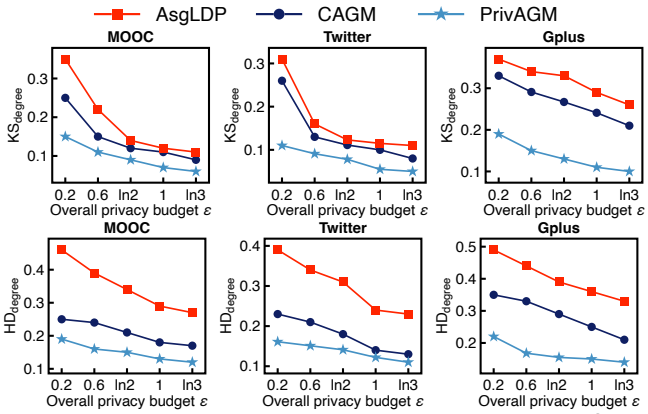


Figure 6. Comparison of PrivAGM and the baselines in terms of $\mathsf{KS}_{\mathsf{degree}}$ and $\mathsf{HD}_{\mathsf{degree}}$, for varying the overall privacy budget ε.

Recall that PrivAGM allocates the entire privacy budget into five parts: $\varepsilon = \varepsilon_E + 2 \cdot \varepsilon_{\mathcal{D}} + 2 \cdot \varepsilon_X + 2 \cdot \varepsilon_F + 2 \cdot \varepsilon_\triangle$, where ε_E for local view encryption, $\varepsilon_{\mathcal{D}}$ for secure degree sequence extraction, ε_X for secure attribute-edge correlation distribution extraction, ε_F for secure attribute distribution extraction, and $\varepsilon_\triangle$ for secure triangle counts extraction. Since certain parts do not introduce loss in utility (e.g., local view encryption), we can straightforwardly set their privacy budgets as 0 during utility evaluation. Hence, our overall privacy budget simplifies to $\varepsilon = \varepsilon_{\mathcal{D}} + \varepsilon_X + \varepsilon_F + 2 \cdot \varepsilon_\triangle$. This results are shown in Table 2, with the overall privacy budget ε set to 1. For simplicity, we only show the results when $\varepsilon_{\mathcal{D}} \geq 0.3, \varepsilon_X \geq 0.1, \varepsilon_F \geq 0.1$, and $\varepsilon_\triangle \geq 0.1$. From the table, it is evident that a larger privacy budget $\varepsilon_{\mathcal{D}}$ corresponds to a smaller $\mathsf{Overall}_{\mathsf{diff}}$, and the optimal privacy budget allocation for all studied datasets is $\varepsilon_{\mathcal{D}} = 0.6, \varepsilon_X = 0.1, \varepsilon_F = 0.1, \varepsilon_\triangle = 0.1$. This is because $\varepsilon_{\mathcal{D}}$ is related to the degree sequence, which is crucial for capturing the structure of the input graph [3].

8.3. Utility Comparison

Degree distribution. Fig. 6 presents a comparison between PrivAGM and the baselines in terms of $\mathsf{KS}_{\mathsf{degree}}$ and $\mathsf{HD}_{\mathsf{degree}}$, for varying the overall privacy budget $\varepsilon \in \{0.2, 0.6, \ln 2, 1, \ln 3\}$. It is evident that AsgLDP achieves the largest $\mathsf{KS}_{\mathsf{degree}}$ and $\mathsf{HD}_{\mathsf{degree}}$ since it is based on LDP, which introduces larger noise than central DP under the same privacy budget. In contrast, our PrivAGM achieves the smallest $\mathsf{KS}_{\mathsf{degree}}$ and $\mathsf{HD}_{\mathsf{degree}}$. This is attributed to PrivAGM effectively preserving the inherent directionality of the edges in the input directed graphs.

Attribute-edge correlation distribution. Fig. 7 shows a comparison between PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{corr}}, \mathsf{HD}_{\mathsf{corr}}$, for varying the privacy budget. PrivAGM consistently achieves the smallest $\mathsf{MREcorr}, \mathsf{HDcorr}$ across all privacy budgets and datasets. Moreover, as ε decreases, the difference in $\mathsf{MREcorr}, \mathsf{HDcorr}$ between PrivAGM and the baselines becomes more significant. This is because PrivAGM has an optimized privacy budget allocation strat-

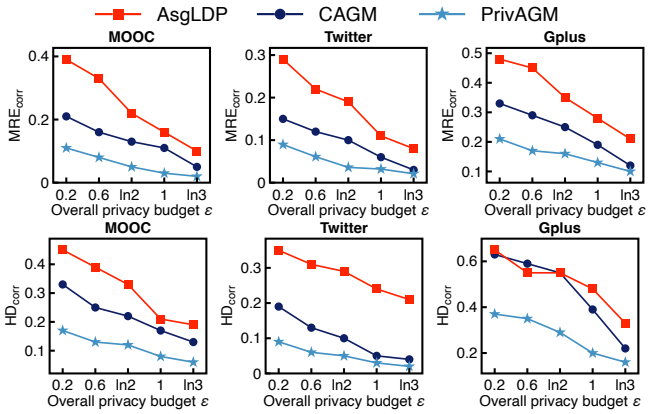


Figure 7. Comparison of PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{corr}}$ and $\mathsf{HD}_{\mathsf{corr}}$, for varying the overall privacy budget ε.

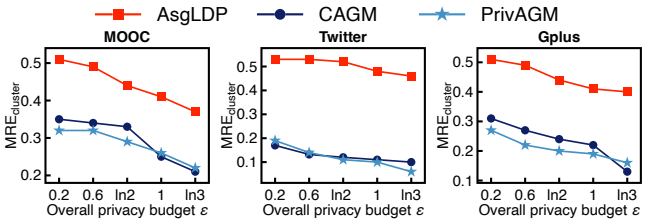


Figure 8. Comparison of PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{cluster}}$, for varying the overall privacy budget ε.

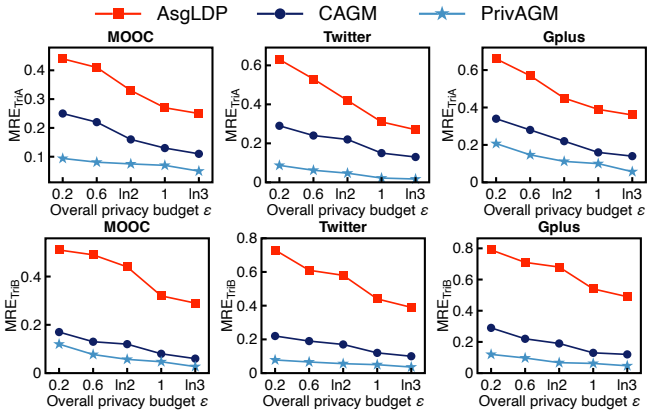


Figure 9. Comparison of PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{TriA}}$ and $\mathsf{HD}_{\mathsf{TriB}}$, for varying the overall privacy budget ε.

egy. Hence, the utility of the synthetic graphs generated by PrivAGM is less affected by ε.

Local clustering coefficient. Fig. 8 presents a comparison between PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{cluster}}$, for varying the overall privacy budget. It can be observed that the $\mathsf{MRE}_{\mathsf{cluster}}$ of PrivAGM and CAGM is approximately similar. In certain instances (e.g., $\varepsilon = \ln 3$ on the Gplus dataset), the $\mathsf{MRE}_{\mathsf{cluster}}$ of CAGM is even lower than that of PrivAGM. This disparity is because the $\mathsf{MRE}_{\mathsf{cluster}}$ fails to capture the difference in edge directionality between the synthetic graphs and the input graphs. Hence, the advantage of PrivAGM in capturing the inherent directionality of the input graphs cannot be fully reflected by the $\mathsf{MRE}_{\mathsf{cluster}}$.

Triangle counts. Fig. 9 presents a comparison between PrivAGM and the baselines in terms of $\mathsf{MRE}_{\mathsf{TriA}}, \mathsf{MRE}_{\mathsf{TriB}}$, for varying the overall privacy budget. The results demonstrate that PrivAGM consistently achieves a significantly small $\mathsf{MRE}_{\mathsf{TriA}}, \mathsf{MRE}_{\mathsf{TriB}}$ (even smaller than 0.1 in most instances) compared to the baselines and validate effectiveness of our sampling-based secure triangle counting protocol. In contrast, AsgLDP consistently exhibits the highest $\mathsf{MRE}_{\mathsf{TriA}}, \mathsf{MRE}_{\mathsf{TriB}}$ (even exceeding 0.5 in most instances). This is because AsgLDP does not optimize the structure of the synthetic graphs by incorporating triangle counts. While CAGM optimizes the structure of the synthetic graphs by incorporating triangle counts, it still cannot achieve lower $\mathsf{MRE}_{\mathsf{TriA}}, \mathsf{MRE}_{\mathsf{TriB}}$ than PrivAGM. This is because CAGM does not consider the inherent directionality of the edges.

8.4. System Cost Evaluation

We provide the system cost evaluation in Appendix D.

9. Related Work

9.1. Private Centralized Graph Analysis

In the *centralized* setting, there exists a considerable body of work on developing methods for private graph analysis. In this context, a curator possesses the complete graph and publishes some noisy information in a manner that satisfies DP. Some works have been dedicated to publishing differentially private statistics of the input graph, e.g., degree distribution [39] and subgraph counting [40]. Other works [3], [7], [16] have focused on developing differentially private generative graph models. Specifically, the work [16] focuses on *unattributed* graphs and only capture the structural information of the input graphs. In contrast, the works [3], [7] center on *attributed* graphs and consider capturing the attribute-edge correlation of the input graphs. Chen *et al.* [7] propose CAGM, an extension of [3], which additionally captures the community structure of the input graph.

However, since the above methods are designed for the centralized setting, they are not applicable in our decentralized setting. Moreover, the mentioned methods focus exclusively on *undirected* graphs, making them unsuitable for achieving satisfactory utility on directed graphs, as demonstrated in our experiments.

9.2. Private Decentralized Graph Analysis

Another line of works considers the more challenging *decentralized* setting, where the graph is presented in a decentralized manner, and no party has direct access to the complete graph. Several works [4], [6], [9], [41] focus on extracting differentially private subgraph counts. Other works [5], [10], [12], [17] have focused on constructing differentially private generative graph models. Specifically, the works [5], [17] aim to construct generative graph models over unattributed and unweighted graphs in a manner that satisfies edge LDP. The work [12] aims to construct differentially private generative graph models over unattributed and weighted graphs. However, these methods [5], [12], [17] fail to capture the attributed information of the graph. Wei *et al.* [10] propose AsgLDP, which considers decentralized *attributed* graphs by incorporating the attribute-edge correlation into the framework of [5]. AsgLDP is the work most related to ours. However, since AsgLDP does not consider the inherent directionality of the edges in the input graphs, its generated synthetic graphs have poor utility when dealing with directed graphs, as demonstrated in our experiments.

10. Conclusion

This paper presents PrivAGM, the first solution for the secure construction of differentially private dAGM on decentralized social graphs. By ingeniously bridging the gap between edge DP, edge LDP, MPC, and generative graph models, PrivAGM enables aggregators to effectively construct differentially private dAGM on decentralized social graphs without compromising the individual privacy of the users. The evaluation results on three real-world graph datasets demonstrate that PrivAGM outperforms the state-of-the-art baselines, offering significantly higher utility in synthetic graphs. We believe that this work lays a solid foundation for advancing research in the secure construction of generative graph models on decentralized social graphs and the synthesis of directed attributed graphs.

References

[1] D. Doran, "Triad-based role discovery for large social systems," in *Proc. of SocInfo*, 2014.

[2] U. Can and B. Alatas, "A new direction in social network analysis: Online social network analysis problems and applications," *Physica A: Statistical Mechanics and its Applications*, vol. 535, p. 122372, 2019.

[3] Z. Jorgensen, T. Yu, and G. Cormode, "Publishing attributed social graphs with formal privacy guarantees," in *Proc. of ACM SIGMOD*, 2016.

[4] H. Sun, X. Xiao, I. Khalil, Y. Yang, Z. Qin, W. H. Wang, and T. Yu, "Analyzing subgraph statistics from extended local views with decentralized differential privacy," in *Proc. of ACM CCS*, 2019.

[5] Z. Qin, T. Yu, Y. Yang, I. Khalil, X. Xiao, and K. Ren, "Generating synthetic decentralized social graphs with local differential privacy," in *Proc. of ACM CCS*, 2017.

[6] J. Imola, T. Murakami, and K. Chaudhuri, "Locally differentially private analysis of graph statistics," in *Proc. of USENIX Security*, 2021.

[7] X. Chen, S. Mauw, and Y. Ramírez-Cruz, "Publishing community-preserving attributed social graphs with a differential privacy guarantee," *Proceedings on Privacy Enhancing Technologies*, vol. 2020, no. 4, pp. 131–152, 2020.

[8] Diaspora, "The online social world where you are in control," online at https://diasporafoundation.org, 2023, [Online; Accessed 1-Jul-2023].

[9] J. Imola, T. Murakami, and K. Chaudhuri, "Communication-efficient triangle counting under local differential privacy," in *Proc. of USENIX Security*, 2022.

[10] C. Wei, S. Ji, C. Liu, W. Chen, and T. Wang, "AsgLDP: collecting and generating decentralized attributed graphs with local differential privacy," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 3239–3254, 2020.

[11] J. Imola, T. Murakami, and K. Chaudhuri, "Differentially private triangle and 4-cycle counting in the shuffle model," in *Proc. of ACM CCS*, 2022.

[12] F. T. Brito, V. A. Farias, C. Flynn, S. Majumdar, J. C. Machado, and D. Srivastava, "Global and local differentially private release of count-weighted graphs," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–25, 2023.

[13] I. Dinur and K. Nissim, "Revealing information while preserving privacy," in *Proc. of ACM PODS*, 2003.

[14] C. Dwork, "Differential privacy," in *Proc. of ICALP*, 2006.

[15] M. G. Hay, *Enabling accurate analysis of private network data*, 2010.

[16] Q. Yuan, Z. Zhang, L. Du, M. Chen, P. Cheng, and M. Sun, "PrivGraph: Differentially private graph data publication by exploiting community information," in *Proc. of USENIX Security*, 2023.

[17] Y. Zhang, J. Wei, X. Zhang, X. Hu, and W. Liu, "A two-phase algorithm for generating synthetic graph under local differential privacy," in *Proc. of ACM ICCNS*, 2018.

[18] J. J. Pfeiffer III, S. Moreno, T. La Fond, J. Neville, and B. Gallagher, "Attributed graph models: Modeling network structure with correlated attributes," in *Proc. of ACM WWW*, 2014.

[19] D. Demmler, T. Schneider, and M. Zohner, "ABY - A framework for efficient mixed-protocol secure two-party computation," in *Proc. of NDSS*, 2015.

[20] M. Curtiss, I. Becker, T. Bosman, S. Doroshenko, L. Grijincu, T. Jackson, S. Kunnatur, S. B. Lassen, P. Pronin, S. Sankar, G. Shen, G. Woss, C. Yang, and N. Zhang, "Unicorn: A system for searching the social graph," *Proc. VLDB Endowment*, vol. 6, no. 11, pp. 1150–1161, 2013.

[21] T. Humphries, R. Akhavan Mahdavi, S. Veitch, and F. Kerschbaum, "Selective MPC: Distributed computation of differentially private key-value statistics," in *Proc. of ACM CCS*, 2022.

[22] S. P. Kasiviswanathan, H. K. Lee, K. Nissim, S. Raskhodnikova, and A. D. Smith, "What can we learn privately?" *SIAM Journal on Computing*, vol. 40, no. 3, pp. 793–826, 2011.

[23] X. He, A. Machanavajjhala, C. J. Flynn, and D. Srivastava, "Composing differential privacy and secure computation: A case study on scaling private record linkage," in *Proc. of ACM CCS*, 2017.

[24] S. Tan, B. Knott, Y. Tian, and D. J. Wu, "CryptGPU: Fast privacy-preserving machine learning on the GPU," in *Proc. of IEEE S&P*, 2021.

[25] E. Dauterman, M. Rathee, R. A. Popa, and I. Stoica, "Waldo: A private time-series database from function secret sharing," in *Proc. of IEEE S&P*, 2022.

[26] S. Tan, W. Chen, R. Deng, and R. A. Popa, "MPCAuth: Multi-factor authentication for distributed-trust systems," in *Proc. of IEEE S&P*, 2023.

[27] MPCVault, "Multisig crypto wallet for business," online at https://mpcvault.com, 2023, [Online; Accessed 1-Jun-2023].

[28] Fireblocks, "Remove the complexity of working with digital assets," online at https://www.fireblocks.com, 2023, [Online; Accessed 1-Jun-2023].

[29] Meta, "The value of secure multi-party computation," online at https://privacytech.fb.com/multi-party-computation/, 2023, [Online; Accessed 1-Jun-2023].

[30] S. L. Warner, "Randomized response: A survey technique for eliminating evasive answer bias," *Journal of the American Statistical Association*, vol. 60, no. 309, pp. 63–69, 1965.

[31] J. Blocki, A. Blum, A. Datta, and O. Sheffet, "Differentially private data analysis of social networks via restricted sensitivity," in *Proc. of ACM ITCS*, 2013.

[32] E. Boyle, N. Chandran, N. Gilboa, D. Gupta, Y. Ishai, N. Kumar, and M. Rathee, "Function secret sharing for mixed-mode and fixed-point secure computation," in *Proc. of EUROCRYPT*, 2021.

[33] S. Eskandarian and D. Boneh, "Clarion: Anonymous communication from multiparty shuffling protocols," in *Proc. of NDSS*, 2022.

[34] R. Eriguchi, A. Ichikawa, N. Kunihiro, and K. Nuida, "Efficient noise generation protocols for differentially private multiparty computation," *IEEE Transactions on Dependable and Secure Computing*, 2022.

[35] Anonymity, "The full version of PrivAGM," https://anonymous.4open.science/api/repo/PrivAGM/file/PrivAGM_full_version.pdf, 2023.

[36] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "ABY2.0: Improved mixed-protocol secure two-party computation." in *Proc. of USENIX Security*, 2021.

[37] F. Chung and L. Lu, "The average distances in random graphs with given expected degrees," *Proceedings of the National Academy of Sciences*, vol. 99, no. 25, pp. 15 879–15 882, 2002.

[38] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world'networks," *nature*, vol. 393, no. 6684, pp. 440–442, 1998.

[39] S. Raskhodnikova and A. Smith, "Lipschitz extensions for node-private graph statistics and the generalized exponential mechanism," in *Proc. of IEEE FOCS*, 2016.

[40] X. Ding, S. Sheng, H. Zhou, X. Zhang, Z. Bao, P. Zhou, and H. Jin, "Differentially private triangle counting in large graphs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 11, pp. 5278–5292, 2021.

[41] Q. Ye, H. Hu, M. H. Au, X. Meng, and X. Xiao, "Lf-gdpr: A framework for estimating graph metrics with local differential privacy," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 10, pp. 4905–4920, 2022.

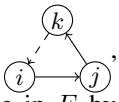
[42] Y. Lindell, "How to simulate it - A tutorial on the simulation proof technique," in *Tutorials on the Foundations of Cryptography*, 2017, pp. 277–346.

Algorithm 5 Refining Graph Structure with Edge Rewiring

Input: The edge set E; $\widetilde{\mathcal{D}}^{\pm}$; $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$.
Output: The new edge set E' after rewiring.

1: $n^A_\triangle, n^B_\triangle = \mathsf{countTriangle}(E)$. ▷Count triangles on E.
2: **while** $n^A_\triangle < \widetilde{n}^A_\triangle$ and $n^B_\triangle < \widetilde{n}^B_\triangle$ **do**
3: //Construct TriA:
4: $i = \mathsf{Sample}([1, N], \widetilde{\mathcal{D}}^{\pm})$. ▷Sample a vertex based on $\widetilde{\mathcal{D}}^{\pm}$.
5: $j = \mathsf{SampleNbr}(E, i)$. ▷Randomly sample a vertex from vertex i's out-neighbors.
6: $k = \mathsf{SampleNbr}(E, j)$.
7: **if** $(k, i) \notin E$ **then**
8: $(q, r) = \mathsf{oldestEdge}(E)$. ▷Get the oldest edge from E.
9: **if** Replacing (q, r) by (k, i) boosts triangle counts **then**
10: $E.\mathsf{replace}((q, r), (k, i))$. ▷Replace (q, r) by (k, i).
11: Make edge (k, i) the youngest edge in E.
12: $n^A_\triangle, n^B_\triangle = \mathsf{countTriangle}(E)$.
13: **else**
14: Make edge (q, r) the youngest edge in E.
15: **end if**
16: **end if**
//Construct TriB:
17: $i = \mathsf{Sample}([1, N], \widetilde{\mathcal{D}}^{\pm})$.
18: $j, k = \mathsf{Sample2Nbr}(E, i)$. ▷Randomly sample two vertices from vertex i's out-neighbors.
19: If $(j, k) \notin E$, replacing the oldest edge by (j,k) as above.
20: **end while**
21: **return** The edge set $E' = E$ after rewiring.

Appendix A. Refining Graph Structure with Edge Rewiring

Algorithm 5 operates on the initial edge set $E^{(0)}$ generated by the directed CL model. It iteratively rewires the edges until the counts of TriA and TriB formed by the edges surpass $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$, respectively. Firstly, in line 1, it counts TriA and TriB on E, and the counts are denoted as $n^A_\triangle$ and $n^B_\triangle$, respectively. Next, in the main loop (lines 2-20), it rewires the edges, alternating between constructing TriA and TriB, until their counts surpass $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$, respectively. Specifically, to create a new edge for constructing TriA (lines 4-16), the algorithm proceeds as follows: 1) sample a vertex i from $[1, N]$ where each vertex $i \in [1, N]$ is sampled with a probability of $\frac{d^+_i}{\sum_{i\in[1,N]} d^+_i}$; 2) randomly sample a vertex j from vertex i's out-neighbors; 3) randomly sample a vertex k from vertex j's out-neighbors. If $(k, i) \notin E$, indicating that vertices i, j, and k form an incomplete TriA, i.e., 

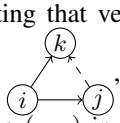
, the algorithm attempts to replace the oldest edge in E by (k, i) to construct TriA (lines 8-15). Specifically, if replacing the oldest edge (q, r) in E with (k, i) would increase both the counts of TriA and TriB, (q, r) is replaced by (k, i) and (k, i) is set as the youngest edge in E. Otherwise, (q, r) is set as the youngest edge in E.

Similarly, Algorithm 5 creates a new edge to construct TriB as follows (lines 17-19). First, a vertex i is sampled from $[1, N]$ where each vertex $i \in [1, N]$ is chosen with a probability of $\frac{d^+_i}{\sum_{i\in[1,N]} d^+_i}$. Then, two vertices j and k are

Algorithm 6 Directed Attributed Graphs Generation

Input: The dAGM parameters $\widetilde{\mathcal{D}}^{\pm}$, $\widetilde{\Theta}_{\vec{X}}$, $\widetilde{\Theta}_F$, $\widetilde{n}^A_{\triangle}$, and $\widetilde{n}^B_{\triangle}$.
Output: The synthetic directed social graph $G = (E, \{f_i\})$.

1: $\{f_i\}_{i\in[1,N]} = \mathsf{Sample}(\widetilde{\Theta}_F)$. ▷Sample a vertex attribute set.
2: $E^{(0)} = \mathsf{directedCL}(\widetilde{\mathcal{D}}^{\pm})$. ▷Sample the initial edge set.
3: $E'^{(0)}$=$\mathsf{Rewire}(E^{(0)}, \widetilde{\mathcal{D}}^{\pm}, \widetilde{n}^A_{\triangle}, \widetilde{n}^B_{\triangle})$. ▷Rewire the edges.
4: Initialize the round $\tau = 0$.
5: **while** edge acceptance probability Θ_{Ac} does not converage **do**
6: Compute the attribute-edge correlation distribution $\widetilde{\Theta}^{(\tau)}_{\vec{X}}$ of the current graph $(E'^{(\tau)}, \{f_i\})$.
7: $R(f\to f') = \frac{\widetilde{\Theta}_{\vec{X}}(f\to f')}{\widetilde{\Theta}^{(\tau)}_{\vec{X}}(f\to f')}, f\to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$.
8: **if** $\tau > 0$ **then**
9: $R(f\to f')$=$R(f\to f')\cdot\Theta^{(\tau-1)}_{Ac}(f\to f'), f\to f' \in \mathbb{Z}_{2^L}\times\mathbb{Z}_{2^L}$.
10: **end if**
11: $\Theta^{(\tau)}_{Ac}(f\to f') = \frac{R(f\to f')}{\mathsf{SUP}(R)}, f\to f' \in \mathbb{Z}_{2^L} \times \mathbb{Z}_{2^L}$.
//Sample a new edge set $E'^{(\tau+1)}$ based on $\Theta^{(\tau)}_{Ac}$:
12: $E^{(\tau+1)} = \mathsf{directedCL}(\widetilde{\mathcal{D}}^{\pm}, \{f_i\}, \Theta^{(\tau)}_{Ac})$.
13: $E'^{(\tau+1)} = \mathsf{Rewire}(E^{(\tau+1)}, \widetilde{\mathcal{D}}^{\pm}, \widetilde{n}^A_{\triangle}, \widetilde{n}^B_{\triangle}, \{f_i\}, \Theta^{(\tau)}_{Ac})$.
14: $\tau = \tau + 1$.
15: **end while**
16: **return** $E = E'^{(\tau)}$; The synthetic social graph $G = (E, \{f_i\})$.

randomly sampled from vertex i's out-neighbors. If $(j, k) \notin E$, indicating that vertices i, j, and k form an incomplete TriB, i.e.,  , the algorithm checks if replacing the oldest edge (q, r) in E with (j, k) would increase both the counts of TriA and TriB. If it does, the algorithm replaces (q, r) with (j, k) and makes (j, k) the youngest edge in E. Otherwise, it makes edge (q, r) the youngest edge in E.

Appendix B.
Putting It All Together

Algorithm 6 presents the end-to-end workflow for synthesizing directed graphs. The vertex attribute set $\{f_i\}_{i\in[1,N]}$ is initially sampled randomly from the attribute distribution $\widetilde{\Theta}_F$ (line 1). The attribute value f_i of each vertex i can be mapped to the attribute vector using the reverse form of Eq. 2. Then, the initial edge set $E^{(0)}$ is sampled using the directed CL model described in Section 6.1 (line 2). Next, Algorithm 5 is utilized to rewire the edges in $E^{(0)}$ to enhance the counts of TriA and TriB formed by the edges greater than $\widetilde{n}^A_{\triangle}$ and $\widetilde{n}^B_{\triangle}$, respectively (line 3). Subsequently, the main loop (lines 5-15) generates new edge sets iteratively by incorporating the accept/reject sampling technique into the directed CL model and Algorithm 5. Specifically, lines 6-11 calculate the new edge acceptance probabilities $\Theta^{(\tau)}_{Ac}$ by measuring the attribute-edge correlations between $\widetilde{\Theta}^{(\tau)}_{\vec{X}}$ in the current graph $(E'^{(\tau)}, \{f_i\})$ and $\widetilde{\Theta}_{\vec{X}}$. $\Theta^{(\tau-1)}_{Ac}$ is the edge acceptance probabilities from the previous round, and $\mathsf{SUP}(R)$ is the least upper bound of set R. Then the new edge set $E'^{(\tau+1)}$ is sampled using the modified structural model (lines 12 and 13). Specifically, when the directed CL model or Algorithm 5 add an edge (i, j) to the new edge set, they initially sample a value uniformly from the range of 0 to 1 ($\mathsf{Sample}(0, 1)$). (i, j) is included in the new edge set (i.e., "accepted") only when $\mathsf{Sample}(0, 1) \leq \Theta^{(\tau+1)}_{Ac}(f_i \to f_j)$, where f_i and f_j are the attributes sampled for vertex i and j in line 1. If $\mathsf{Sample}(0, 1) > \Theta^{(\tau+1)}_{Ac}(f_i\to f_j)$, (i, j) is not included in the new edge set (i.e., "rejected").

Appendix C.
Privacy and Security Analysis

Since the users can only observe their local views and the final output of the differentially private computation (received from the aggregators), we will specifically focus on the situation where the adversary controls one of $P_{\{1,2,3\}}$.

C.1. Leakage Functions

We define the leakage functions of PrivAGM as a tuple $\mathcal{L} = (\mathcal{L}_E, \mathcal{L}_{\mathcal{D}}, \mathcal{L}_X, \mathcal{L}_F, \mathcal{L}_{\triangle})$ formed as follows:

- $\mathcal{L}_E = \{[E_i]\}$, where $[E_i] = \{(i, j, \langle b_{ij}\rangle)\}$ denotes the local neighbor list held by the adversary for user i.
- $\mathcal{L}_{\mathcal{D}} = ([\hat{\mathbf{T}}'_{\mathcal{D}}], \widetilde{\mathbf{H}}_{\mathcal{D}})$, where $[\hat{\mathbf{T}}'_{\mathcal{D}}]$ denotes the table revealed in line 16 of Algorithm 3, and $\widetilde{\mathbf{H}}_{\mathcal{D}}$ denotes the histogram revealed in line 22 of Algorithm 3. While the adversary acquires additional information during the execution of the remaining operations in Algorithm 3, since this information is derived from $\widetilde{\mathbf{H}}_{\mathcal{D}}$, it is not included in $\mathcal{L}_{\mathcal{D}}$.
- $\mathcal{L}_X = ([\hat{\mathbf{T}}'_X], \widetilde{\mathbf{H}}_X)$, where $[\hat{\mathbf{T}}'_X]$ and $\widetilde{\mathbf{H}}_X$ are the table and histogram revealed in secure attribute-edge correlation distribution extraction, respectively.
- $\mathcal{L}_F$=$([\hat{\mathbf{T}}'_F], \widetilde{\mathbf{H}}_F)$, where $[\hat{\mathbf{T}}'_F]$ and $\widetilde{\mathbf{H}}_F$ are the table and histogram revealed in secure attribute distribution extraction.
- $\mathcal{L}_{\triangle} = (\widetilde{n}^A_{\triangle}, \widetilde{n}^B_{\triangle})$, which are the triangle counts revealed in line 22 of Algorithm 4.

Since the generation of directed attributed graphs, solely relies on $\mathcal{L}$, the involved information is excluded from $\mathcal{L}$.

C.2. Privacy Analysis

***Theorem* 1.** *The local neighbor list $[E_i] \in \mathcal{L}_E$ of user $i \in [1, N]$ held by the adversary adheres to pure ε_E-edge LDP with $\varepsilon_E = \max\{\ln(1/p), \ln(3 - 2p)\}$.*

Proof. Note that the secret shares $\{\langle b_{ij}\rangle\}$ contained within $[E_i]$ reveal nothing about the corresponding values. Hence, $[E_i]$ can be described as an observed N-dimensional bit vector $\mathbf{b} = (b_1, \cdots, b_N)$ for the adversary, where $b_j = 1, j \in [1, N]$ if $(i, j, \langle b_{ij}\rangle) \in [E_i]$, and otherwise $b_j = 0$.

We abuse $\mathcal{M}$ to denote the secure mechanisms in PrivAGM, and use $Pr[\mathbf{A}[i, j] \to b_j]$ to represent the probability that, given a bit $\mathbf{A}[i, j]$ of user i's local neighbor list, the adversary observes $b_j \in \{0, 1\}$. The values of $Pr[\mathbf{A}[i, j] \to b_j]$ can be categorized into four cases:

- $\mathbf{A}[i,j] = 0$ and $b_j = 0$: This case corresponds to the scenario where either user i does not include the zero bit $\mathbf{A}[i, j]$ in E_i (with a probability of $1-p$), or user i includes $\mathbf{A}[i, j]$ in E_i (with a probability of p), but does not choose the adversary to receive $(i, j, \langle 0\rangle)$ (with a probability of $1/3$). Therefore, $Pr[0 \to 0] = 1 - p + p \cdot 1/3 = 1 - 2p/3$.
- $\mathbf{A}[i,j] = 1$ and $b_j = 0$: This case corresponds to the scenario where user i includes the nonzero bit $\mathbf{A}[i, j]$ in E_i (with a probability of 1), but does not choose the adversary to receive $(i, j, \langle 1\rangle)$ (with a probability of $1/3$). Therefore, $Pr[1 \to 0] = 1 \cdot 1/3 = 1/3$.

• $\mathbf{A}[i,j] = 1$ and $b_j = 1$: This case corresponds to the scenario where user i includes the nonzero bit $\mathbf{A}[i,j]$ in E_i (with a probability of 1) and also chooses the adversary to receive $(i, j, \langle 1 \rangle)$ (with a probability of 2/3). Therefore, $Pr[1 \to 1] = 1 \cdot 2/3 = 2/3$.

• $\mathbf{A}[i,j] = 0$ and $b_j = 1$: This case corresponds to the scenario where user i includes the zero bit $\mathbf{A}[i,j]$ in E_i (with a probability of p) and also chooses the adversary to receive $(i, j, \langle 0 \rangle)$ (with a probability of 2/3). Therefore, $Pr[0 \to 1] = p \cdot 2/3 = 2p/3$.

Given any two local neighbor lists $\mathbf{A}[i,:]$ and $\mathbf{A}'[i,:]$ that differ in one bit, we need to prove that $\frac{Pr[\mathcal{M}(\mathbf{A}[i,:])=\mathbf{b}]}{Pr[\mathcal{M}(\mathbf{A}'[i,:])=\mathbf{b}]} \leq e^{\varepsilon_E}$. Without loss of generality, let us assume that $\mathbf{A}[i,1]$ and $\mathbf{A}'[i,1]$ are not identical. Then we have:

$$\frac{Pr[\mathcal{M}(\mathbf{A}[i,:])=\mathbf{b}]}{Pr[\mathcal{M}(\mathbf{A}'[i,:])=\mathbf{b}]} = \frac{Pr[\mathbf{A}[i,1]\to b_1]\cdots Pr[\mathbf{A}[i,N]\to b_N]}{Pr[\mathbf{A}'[i,1]\to b_1]\cdots Pr[\mathbf{A}'[i,N]\to b_N]} = \frac{Pr[\mathbf{A}[i,1]\to b_1]}{Pr[\mathbf{A}'[i,1]\to b_1]}. \tag{3}$$

We analyze Eq. 3 in four cases:
1) $\frac{Pr[0\to0]}{Pr[1\to0]} = \frac{1-2p/3}{1/3} = 3-2p$; 2) $\frac{Pr[1\to0]}{Pr[0\to0]} = \frac{1/3}{1-2p/3} = \frac{1}{3-2p}$; 3) $\frac{Pr[0\to1]}{Pr[1\to1]} = \frac{2p/3}{2/3} = p$; 4) $\frac{Pr[1\to1]}{Pr[0\to1]} = \frac{2/3}{2p/3} = 1/p$.
We then need to find the upper bound (i.e., e^{ε_E}) of the set $\{3-2p, \frac{1}{3-2p}, p, 1/p\}$. Since $0 \leq p \leq 1$, we have $1/p \geq p$ and $3-2p \geq \frac{1}{3-2p}$. Therefore, we can conclude that $\{3-2p, \frac{1}{3-2p}, p, 1/p\} \leq \max\{1/p, 3-2p\} = e^{\varepsilon_E}$, and $\varepsilon_E = \max\{\ln(1/p), \ln(3-2p)\}$. The probability that the adversary views the same $\mathbf{b}$ from $\mathbf{A}[i,:], \mathbf{A}'[i,:]$ is bounded by ε_E, which satisfies ε_E-edge LDP based on Def. 1. □

Theorem 2. *Given $\mathcal{L}_\mathcal{D} = ([\hat{\mathbf{T}}'_\mathcal{D}], \widetilde{\mathbf{H}}_\mathcal{D})$, in the view of adversary, $[\hat{\mathbf{T}}'_\mathcal{D}]$ adheres to $(\varepsilon_\mathcal{D}, \delta_\mathcal{D})$-edge DP and $\widetilde{\mathbf{H}}_\mathcal{D}$ adheres to pure $\varepsilon_\mathcal{D}$-edge DP.*

Proof. Note that only $P_{1,2}$ have access to $[\hat{\mathbf{T}}'_\mathcal{D}]$, and therefore we only analyze the view of the adversary who controls either P_1 or P_2. Since the secret shares (i.e., the 3nd column of table $[\hat{\mathbf{T}}'_\mathcal{D}]$) reveal nothing about the corresponding values, $[\hat{\mathbf{T}}'_\mathcal{D}]$ can be described as a histogram (denoted as $\mathbf{H}_\mathcal{D}$) of degree pairs observed by the adversary. In $\mathbf{H}_\mathcal{D}$, each bin $\mathbf{H}_\mathcal{D}[d^+, d^-], d^+, d^- \in [0, k]$ represents the (noisy) number of vertices that have degree pair (d^+, d^-). Then we proceed to prove that with probability $1-\delta_\mathcal{D}$, the probability for the adversary viewing the same $\mathbf{H}_\mathcal{D}$ from two neighboring graphs G and G' is bounded by $e^{\varepsilon_\mathcal{D}}$.

If all the noise (drawn in line 10 of Algorithm 3) for the degree pairs are non-negative, then the probability to output the same noisy histogram $\mathbf{H}_\mathcal{D}$ from G, G' is $\frac{Pr[\mathcal{M}(G)=\mathbf{H}_\mathcal{D}]}{Pr[\mathcal{M}(G')=\mathbf{H}_\mathcal{D}]}$:

$$\frac{\prod_{d^+,d^-\in[0,k]} Pr[\mathbf{H}_\mathcal{D}[d^+,d^-] - \mathbf{H}_G[d^+,d^-]]}{\prod_{d^+,d^-\in[0,k]} Pr[\mathbf{H}_\mathcal{D}[d^+,d^-] - \mathbf{H}_{G'}[d^+,d^-]]}, \tag{4}$$

where $\mathbf{H}_G[d^+, d^-]$ is the number of vertices in graph G that have degree pair (d^+, d^-). $Pr[\mathbf{H}_\mathcal{D}[d^+,d^-] - \mathbf{H}_G[d^+,d^-]]$ is the probability drawing the noise $\mathbf{H}_\mathcal{D}[d^+,d^-] - \mathbf{H}_G[d^+,d^-]$ from the Laplace distribution $Lap(\varepsilon_\mathcal{D}, \delta_\mathcal{D}, 4)$. Let $D_\triangle$ be the set of bins where $\mathbf{H}_G$ and $\mathbf{H}_G$ are not equal. Then Eq. 4:

$$\begin{aligned}
&\frac{\prod_{(d^+,d^-)\in D_\triangle} e^{\frac{-\varepsilon_\mathcal{D}}{4}\cdot|\mathbf{H}_\mathcal{D}[d^+,d^-]-\mathbf{H}_G[d^+,d^-]-\mu|}}{\prod_{(d^+,d^-)\in D_\triangle} e^{\frac{-\varepsilon_\mathcal{D}}{4}\cdot|\mathbf{H}_\mathcal{D}[d^+,d^-]-\mathbf{H}_{G'}[d^+,d^-]-\mu|}} \\
&= \prod_{(d^+,d^-)\in D_\triangle} e^{\frac{\varepsilon_\mathcal{D}}{4}\cdot\left(|\mathbf{H}_\mathcal{D}[d^+,d^-]-\mathbf{H}_{G'}[d^+,d^-]-\mu| - |\mathbf{H}_\mathcal{D}[d^+,d^-]-\mathbf{H}_G[d^+,d^-]-\mu|\right)} \\
&= e^{\frac{\varepsilon_\mathcal{D}}{4}\cdot\sum_{(d^+,d^-)\in D_\triangle} |\mathbf{H}_\mathcal{D}[d^+,d^-]-\mathbf{H}_{G'}[d^+,d^-]-\mu| - |\mathbf{H}_\mathcal{D}[d^+,d^-]-\mathbf{H}_G[d^+,d^-]-\mu|} \\
&\leq e^{\frac{\varepsilon_\mathcal{D}}{4}\cdot\sum_{(d^+,d^-)\in D_\triangle} |\mathbf{H}_G[d^+,d^-]-\mathbf{H}_{G'}[d^+,d^-]|} \leq e^{\frac{\varepsilon_\mathcal{D}}{4}\cdot 4} = e^{\varepsilon_\mathcal{D}}.
\end{aligned} \tag{5}$$

The above proof is based on a fact that as G and G' differ by the presence or absence of a single edge, the maximum discrepancy in the values of their degree pair histograms is 4, i.e., $\sum_{(d^+,d^-)\in D_\triangle} |\mathbf{H}_G[d^+,d^-] - \mathbf{H}_{G'}[d^+,d^-]| \leq 4$.

The probability to draw a negative noise from $Lap(\varepsilon_\mathcal{D}, \delta_\mathcal{D}, 4)$ is [23]: $Pr[n < 0] = \sum_{x=-1}^{-\infty} \frac{e^{\frac{\varepsilon_\mathcal{D}}{4}}-1}{e^{\frac{\varepsilon_\mathcal{D}}{4}}+1} \cdot e^{\frac{-\varepsilon_\mathcal{D}\cdot|x-\mu|}{4}} = \frac{e^{\frac{-\mu\cdot\varepsilon_\mathcal{D}}{4}}}{e^{\frac{\varepsilon_\mathcal{D}}{4}}+1}$. Given $\mu = -\frac{4}{\varepsilon_\mathcal{D}} \cdot \ln((e^{\frac{\varepsilon_\mathcal{D}}{4}}+1)\cdot(1-(1-\delta_\mathcal{D})^{\frac{1}{4}}))$, we have $Pr[n<0] = 1-(1-\delta_\mathcal{D})^{\frac{1}{4}}$. As there are at most 4 bins in $\mathbf{H}_G, \mathbf{H}_{G'}$ that are not equal. Hence, the overall failing probability is $1-(1-Pr[n<0])^4 = \delta_\mathcal{D}$. This means that with probability $1-\delta_\mathcal{D}$, the probability for the adversary viewing the same $\mathbf{H}_\mathcal{D}$ from G,G' is bounded by $e^{\varepsilon_\mathcal{D}}$, which satisfies $(\varepsilon_\mathcal{D}$,$\delta_\mathcal{D})$-edge DP based on Def. 2.

Then we proceed to prove that the probability for the adversary viewing the same $\widetilde{\mathbf{H}}_\mathcal{D}$ from G, G' is bounded by $e^{\varepsilon_\mathcal{D}}$. The probability to output the same $\widetilde{\mathbf{H}}_\mathcal{D}$ from G and G' is bounded by $\frac{Pr[\mathcal{M}(G)=\widetilde{\mathbf{H}}_\mathcal{D}]}{Pr[\mathcal{M}(G')=\widetilde{\mathbf{H}}_\mathcal{D}]} \leq e^{\varepsilon_\mathcal{D}}$. The proof follows Eq. 5 directly. Since the noises added to $\widetilde{\mathbf{H}}_\mathcal{D}$ (line 21 of Algorithm 3) are drawn from $Lap(\varepsilon_\mathcal{D}, 0, 4)$ (i.e., $\delta = 0$), $\mathcal{M}$ satisfies pure $\varepsilon_\mathcal{D}$-edge DP based on Def. 2. □

Theorem 3. *Given $\mathcal{L}_X = ([\hat{\mathbf{T}}'_X], \widetilde{\mathbf{H}}_X)$, in the view of the adversary, $[\hat{\mathbf{T}}'_X]$ adheres to $(\varepsilon_X, \delta_X)$-edge DP and $\widetilde{\mathbf{H}}_X$ adheres to pure ε_X-edge DP.*

Proof. Since the proof follows the same idea as the proof of Theorem 2, we omit the proof here. □

Theorem 4. *Given $\mathcal{L}_F = ([\hat{\mathbf{T}}'_F], \widetilde{\mathbf{H}}_F)$, in the view of the adversary, $[\hat{\mathbf{T}}'_F]$ adheres to $(\varepsilon_F, \delta_F)$-edge DP and $\widetilde{\mathbf{H}}_F$ adheres to pure ε_F-edge DP.*

Proof. Since the proof also follows the same idea as the proof of Theorem 2, we omit the proof here. □

Theorem 5. *Given $\mathcal{L}_\triangle = (\widetilde{n}^A_\triangle, \widetilde{n}^B_\triangle)$, in the view of the adversary, both $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$ adhere to pure $\varepsilon_\triangle$-edge DP.*

Proof. Since the proof follows the same idea as the proof of Theorem 2, we detail it in our full version [35]. □

C.3. Security Analysis

We employ the simulation paradigm [42] of MPC to establish PrivAGM's security guarantees. Our objective is to ensure that an adversary $\mathcal{A}$ with the ability to *statically* corrupt at most one of $P_{1,2,3}$ cannot obtain any additional knowledge beyond the differentially private output of $\mathcal{L}$. Proving security within the simulation paradigm involves

establishing two distinct worlds: the real world, where the actual protocol is executed by honest parties, and the ideal world, where an ideal functionality $\mathcal{F}$ takes inputs from the parties and directly outputs the result to the relevant party.

Ideal functionality $\mathcal{F}$. We begin by introducing our stateful ideal functionality $\mathcal{F}$: (1) $\mathsf{Colle}(\mathcal{L}_E)$: $\mathcal{F}$ runs the decentralized social graph collection given $\mathcal{L}_E$. (2) $\mathsf{edgeClip}(\mathcal{L}_E, k, w)$: $\mathcal{F}$ runs the efficient edge clipping given $\mathcal{L}_E$, k, and w. (3) $\mathsf{extDeg}(\mathcal{L}_E, \mathcal{L}_\mathcal{D}, k)$: $\mathcal{F}$ runs the degree sequence extraction given $\mathcal{L}_E$, $\mathcal{L}_\mathcal{D}$, and k. (4) $\mathsf{extCorr}(\mathcal{L}_E, \mathcal{L}_X, k)$: $\mathcal{F}$ runs the attribute-edge correlation distribution extraction given $\mathcal{L}_E$, $\mathcal{L}_X$, and k. (5) $\mathsf{extAttr}(\mathcal{L}_F)$: $\mathcal{F}$ runs the attribute distribution extraction given $\mathcal{L}_F$. (6) $\mathsf{extTria}(\mathcal{L}_E, \mathcal{L}_\triangle, k)$: $\mathcal{F}$ runs the triangle counts extraction given $\mathcal{L}_E$, $\mathcal{L}_\triangle$, and k. Given that the generation of directed attributed graphs, is solely dependent on the output of $\mathcal{L}$, its procedure is excluded from $\mathcal{F}$.

***Definition* 4.** *Let $\prod$ denote the protocol for constructing dAGM on a decentralized social graph G. $\mathsf{View}^{\mathsf{Real}}_{\prod(G)}$ represents the view of $\mathcal{A}$ in the real-world experiment. In the ideal world, a probabilistic polynomial-time simulator $\mathcal{S}$ generates a simulated view $\mathsf{View}^{\mathsf{Ideal}}_{\mathcal{S},\mathcal{L}}$ for $\mathcal{A}$ using only the output of $\mathcal{L}$. If $\mathsf{View}^{\mathsf{Ideal}}_{\mathcal{S},\mathcal{L}}$ is indistinguishable from $\mathsf{View}^{\mathsf{Real}}_{\prod(G)}$, we conclude that $\prod$ is secure.*

***Theorem* 6.** *PrivAGM securely evaluates $\mathcal{F}$ based on Def. 4.*

Proof. Since the roles of $P_{\{1,2,3\}}$ in PrivAGM are symmetric, with the exception that $P_{1,2}$ receive the secret shares of dummy entities (i.e., $[\![\mathbf{T}']\!]$) from P_3, it is enough to analyze the existence of $\mathcal{S}$ under the condition that $\mathcal{A}$ corrupts P_1.

Simulator $\mathcal{S}$ in Colle: During this phase, P_1 exclusively receives the set of tuples $\{(i, j, \langle b_{ij}\rangle)\}$. Therefore, we can trivially construct the simulator $\mathcal{S}$ by invoking the simulator of ASS [19] and providing it with the output of $\mathcal{L}_E$ as input.

Simulator $\mathcal{S}$ in edgeClip: The efficient oblivious edge clipping protocol is presented in Algorithm 2. Initially, $\mathcal{A}$ possesses the clipped local neighbor lists $\{(i, j, \langle b_{ij}\rangle)\}$, k, and w. $\mathcal{A}$ only receives messages from other honest aggregators during the execution of lines 7, 10, 20, and 23. The operations in lines 7 and 20 are based on the FSS-based DCF from [32]. Hence, the simulator can be trivially constructed by invoking the simulator of FSS-based DCF [32] and providing it with $k/3$ as input. The operations in lines 10 and 23 rely on the basic secure multiplication operation in the ASS domain. Hence, the simulator can be trivially constructed by invoking the simulator of ASS [19].

Simulator $\mathcal{S}$ in extDeg: The secure degree sequence extraction protocol is given in Algorithm 3. Initially, $\mathcal{A}$ possesses the clipped local neighbor lists $\{(i, j, \langle b_{ij}\rangle)\}$, k, $\varepsilon_\mathcal{D}$, $\delta_\mathcal{D}$, and the secret shares of Laplacian noises. $\mathcal{A}$ only receives messages from other honest aggregators during the execution of lines 5, 13, 15, 16, and 22. The operations in lines 5 and 13 rely on the secret sharing operation in the ASS domain. Hence, the simulator can be trivially constructed by invoking the simulator of ASS [19]. The operation in line 15 is based on the oblivious shuffle protocol from [33]. Therefore, the simulator can be trivially constructed by invoking the simulator of oblivious shuffle [33]. During the execution of lines 16 and 22, $\mathcal{A}$ receives secret shares from P_2 to revel $[\hat{\mathbf{T}}']$ and $\widetilde{\mathbf{H}}$. Based on the security of ASS, the secret shares from P_2 are uniformly random in the view of $\mathcal{A}$. For revealed $[\hat{\mathbf{T}}']$ and $\widetilde{\mathbf{H}}$, $\mathcal{S}$ adjusts P_2' shares such that the revealed values are indeed the coefficient vector it received from $\mathcal{F}$ (i.e., $\mathcal{L}_\mathcal{D}$).

Simulator $\mathcal{S}$ in extCorr and extAttr: As the analysis of the existence of $\mathcal{S}$ in extCorr and extAttr follows the same idea as that in extDeg, we omit the analysis here.

Simulator $\mathcal{S}$ in extTria: The secure triangle counts extraction protocol is given in Algorithm 4. Initially, $\mathcal{A}$ possesses $\{(i, j, \langle b_{ij}\rangle)\}$, k, $\varepsilon_\mathcal{D}$, $\delta_\mathcal{D}$, and the secret shares of Laplacian noises. $\mathcal{A}$ only receives messages from $P_{2,3}$ during the execution of lines 6, 14, and 22. The operations in lines 6 and 14 are based on the secure 3-input multiplication gate from [36]. Hence, the simulator can be trivially constructed by invoking the simulator of the secure 3-input multiplication gate [36]. During the execution of line 22, $\mathcal{A}$ receives secret shares from P_2 and P_3 to revel $\widetilde{n}^A_\triangle$ and $\widetilde{n}^B_\triangle$. Based on the security of ASS, the secret shares from $P_{2,3}$ are uniformly random in the view of $\mathcal{A}$. For revealed $\widetilde{n}^A_\triangle, \widetilde{n}^B_\triangle$, $\mathcal{S}$ adjusts the shares of $P_{2,3}$ such that the revealed values are indeed the coefficient vector it received from $\mathcal{F}$ (i.e., $\mathcal{L}_\triangle$). □

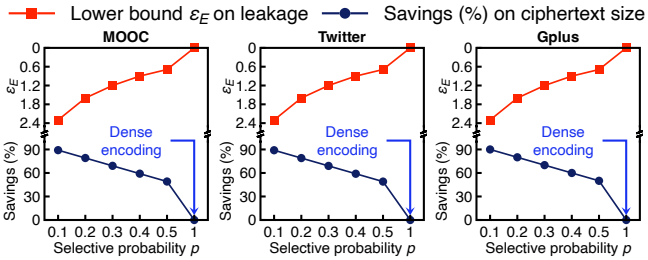


Figure 10. Evaluation of ε_E for leakage in local neighbor list encryption and the savings on the resulting ciphertext size, for varying p.

Appendix D. System Cost Evaluation

Note that we do not compare PrivAGM with the baselines in terms of system cost, as CAGM considers a centralized setting and AsgLDP has significant loss in the generated synthetic graphs. In addition, while PrivAGM consists of three phases, as phase (3) involves generating synthetic graphs in the plaintext domain, which is a rapid process, we do not include the cost of phase (3) in our evaluation.

Local neighbor list encryption. Fig. 10 presents the results of the lower bound ε_E for leakage in local neighbor list encryption and the savings on the resulting ciphertext size, for varying the selective probability $p \in \{0.1, 0.2, 0.3, 0.4, 0.5, 1\}$. The results show that PrivAGM achieves a significant reduction in the resulting ciphertext size compared to direct encryption of the complete local neighbor list in dense encoding (up to 90% under $p = 0.1, \varepsilon_E \approx 2.3$). Moreover, the savings on the resulting ciphertext size remain consistent across different datasets, highlighting the scalability of our local neighbor list encryption protocol. These savings will significantly reduce the system cost in the subsequent secure dAGM construction.

Oblivious edge clipping. Fig. 11 presents a comparison of basic and efficient oblivious edge clipping in

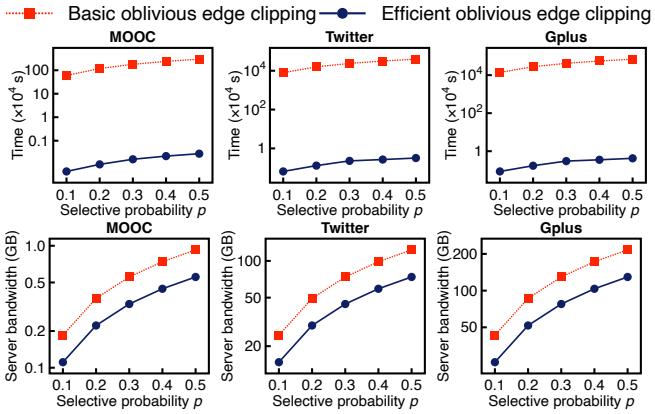


Figure 11. Comparison of basic and efficient oblivious edge clipping in terms of time and communication cost, for varying p.

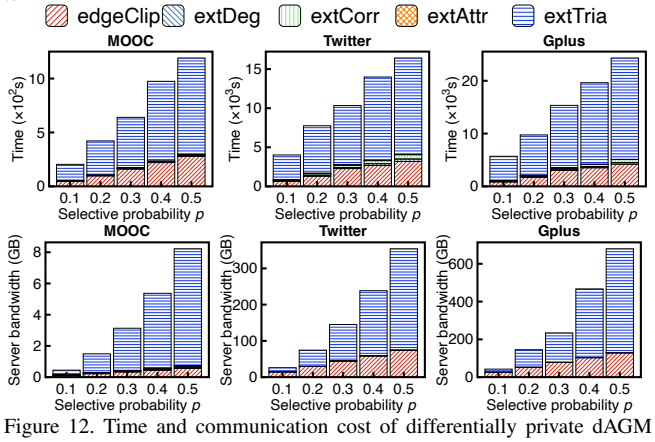


Figure 12. Time and communication cost of differentially private dAGM construction and their breakdown, for varying p.

terms of time and communication cost, for varying $p \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$. The results for the basic protocol are estimated from processing 1000 encrypted edges, as it is significantly slow (taking a few days). The comparison shows that the efficient protocol achieves a substantial speedup compared to the basic protocol (up to 10000×). Additionally, the efficient protocol achieves approximately 50% savings in communication cost compared to the basic protocol. These performance improvements stem from the fact that the efficient protocol allows the aggregators to process the out/in-degree of each vertex in parallel and obliviously clip the edges in batches.

Differentially private dAGM construction. Fig. 12 presents the time and communication cost of differentially private dAGM construction and their breakdown, for varying $p \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$. It can be observed that the majority of the cost can be attributed to extTria. Its high time cost is primarily due to the communication required by the 3-input multiplication gates (i.e., lines 6 and 14 of Algorithm 4). Additionally, the results further demonstrate that our local neighbor list encryption can significantly save the cost of differentially private dAGM construction. While the cost of differentially private dAGM construction is relatively high, it is a *one-off* process, and once the dAGM is learned, it can generate numerous synthetic graphs, making the overall cost affordable in practice.